

Especificación de Requisitos de Software (SRS)

Sistema: SGED — Sistema de Gestión para la Escuela Deportiva ProFútbol **Versión del documento:** 1.6 (Entrega Final, etiqueta `v1.0.2` — revisado el 2026-09-08 tras las correcciones de requisitos M1–M9; el commit defendido es el que apunta la etiqueta (`git rev-parse v1.0.2^{commit}`)). **Estructura:** basada en ISO/IEC/IEEE 29148:2018 **Repositorio:** https://github.com/DarwinSM21/SGED_APPWEB

***Nota de redacción (resuelve OBS-01, Entrega 1A).** El docente observó que los requisitos funcionales estaban redactados como títulos ("Registro de estudiantes") en vez de como requisitos. En este documento **todo requisito funcional se enuncia con la forma "El sistema deberá..."**, con un identificador único, una prioridad y un criterio de verificación comprobable.*

1. Introducción

1.1 Propósito

Este documento especifica los requisitos funcionales y no funcionales de SGED, una aplicación web para la gestión administrativa y deportiva de la escuela de fútbol formativo ProFútbol. Está dirigido al equipo de desarrollo y al docente evaluador del Proyecto Fin de Curso.

1.2 Alcance

SGED cubre cinco dominios:

- Seguridad y acceso** — personas, usuarios, roles, autenticación y restablecimiento de contraseña (RF-01 a RF-07, RF-37; auditoría RF-42).
- Gestión académica/administrativa** — registro y mantenimiento de estudiantes y sus categorías; **cobro de membresías y pagos diarios** (RF-38); **consentimiento y gestión de representantes legales** (RF-39, RF-41) e informes al representante (RF-40); reportes en PDF (RF-43) y exportación de los datos propios (RF-44).
- Dominio deportivo** — entrenadores y su catálogo de especialidades (RF-46), horarios, sesiones de entrenamiento, asistencia y su resumen (RF-47), evaluación diaria del desempeño y partidos.
- Inventario** — catálogo de artículos deportivos (uniformes, balones, implementos), control de stock por movimientos y asignación de artículos a estudiantes o entrenadores (RF-27 a RF-30, ver ADR-003).
- Soporte y observabilidad** — alertas operativas (RF-45), bitácora de auditoría y generación de comentarios con modelo de lenguaje sobre datos seudonimizados (RNF-16).

1.3 Estado de implementación (declaración de honestidad)

***Esta sección resuelve OBS-12 (Entrega 1B),** donde el docente observó que el informe describía funcionalidad que no existía en el repositorio. Para evitar repetir ese error, cada requisito indica explícitamente su estado real, verificable en el código:*

Estado	Significado
 Implementado	Existe endpoint REST funcional, con pruebas y evidencia de ejecución.
 Modelado	El esquema de base de datos existe y está migrado (Flyway), pero aún no se expone vía API REST.
 Planificado	Solo especificado en este documento; sin esquema ni código.

Ningún requisito marcado  o  debe interpretarse como funcionalidad entregada.

1.4 Definiciones y acrónimos

Término	Definición
Categoría	Grupo etario de competencia, definido por un rango de edad (p. ej. "Sub-12"). Desde la reestructuración de paquetes es una entidad propia (deportivo.categorias) con edad_min/edad_max, no un texto libre.
Baja lógica	Marcar un registro como inactivo (activo = FALSE) sin borrarlo físicamente.
JWT	JSON Web Token (RFC 7519), credencial de sesión firmada.
JTI	Identificador único de un JWT, usado para revocarlo.
RFID	Identificación por radiofrecuencia; medio previsto para marcar asistencia.
ProblemDetail	Formato de respuesta de error de RFC 7807 / RFC 9457.
Representante	Padre, madre o tutor legal de un estudiante menor de edad.

1.5 Referencias

- ISO/IEC/IEEE 29148:2018 — Requirements engineering.
- ISO/IEC 25010:2011 — Modelo de calidad de producto software.
- RFC 9110 — HTTP Semantics.
- RFC 7519 — JSON Web Token.
- RFC 9457 — Problem Details for HTTP APIs.
- OWASP Top 10:2021.

2. Descripción general

2.1 Perspectiva del producto

SGED es un sistema cliente-servidor de tres capas:

- **Frontend:** Angular (SPA), servido por nginx con terminación TLS en `:8443` .
- **Backend:** API REST en Spring Boot 3.2 (Java 21), puerto `:8080` .
- **Persistencia:** PostgreSQL 16 (esquemas `seguridad` , `academico` , `deportivo` e `inventario`) y Redis 7 (caché y lista de revocación de tokens).

Orquestación reproducible vía Docker Compose con imágenes fijadas por digest SHA-256.

2.2 Actores del sistema

Actor	Descripción	Rol técnico
Administrador	Gestiona usuarios, estudiantes y configuración. Único actor con permisos de escritura sobre estudiantes.	ADMINISTRADOR
Entrenador	Consulta estudiantes de sus categorías, registra asistencia y evaluación diaria.	ENTRENADOR

Recepcionista	Registra estudiantes, cobra membresías/pagos diarios y emite el QR de asistencia.	RECEPCIONISTA
Representante	Tutor legal del estudiante; consulta informes de sus representados.	REPRESENTANTE
Estudiante	Marca su propia asistencia escaneando el QR.	ESTUDIANTE

No existe un rol genérico de "usuario estándar": cada cuenta se crea con uno de estos roles reales (`rol` es obligatorio en `POST /api/auth/registro`). Los cinco están sembrados en `db/seed.sql` y son los que evalúan las anotaciones `@PreAuthorize` del código.

2.3 Restricciones de diseño

- **RD-01.** El sistema deberá ejecutarse íntegramente mediante contenedores Docker, sin instalación manual de dependencias en la máquina anfitriona.
- **RD-02.** Las operaciones elementales (CRUD simple, consultas paginadas) deberán resolverse con Spring Data JPA; las operaciones de agregación y actualización masiva con criterio de negocio deberán ejecutarse en el motor de base de datos mediante procedimientos almacenados versionados.
- **RD-03.** El sistema no deberá construir sentencias SQL por concatenación dinámica de cadenas en ninguna capa.

3. Requisitos funcionales

3.1 Módulo de seguridad y acceso

RF-01 — Registro de usuarios *El sistema deberá permitir que un usuario con rol ADMINISTRADOR registre nuevas cuentas de usuario, asociándolas a una persona y a uno o más roles.*

- **Prioridad:** Alta · **Estado:** ☒ Implementado · **MoSCoW:** Must
- **Método de verificación:** Test
- **Origen:** `POST /api/auth/registro` — `AuthController.java:63`
- **Restricción de acceso:** `@PreAuthorize("hasRole('ADMINISTRADOR')")`
- **Verificación:** un usuario no autenticado o sin rol ADMINISTRADOR deberá recibir `401 / 403` . Prueba: `AuthServiceTest.registroExitoso` , `AuthServiceTest.registroEmailDuplicado` . Evidencia OWASP A01: `docs/mediciones/sec/a01-acceso-roto.txt` .

RF-02 — Autenticación de usuarios *El sistema deberá autenticar a un usuario mediante nombre de usuario y contraseña, y deberá emitir la credencial de sesión exclusivamente en una cookie `HttpOnly` , `Secure` y `SameSite=Strict` , sin exponer el token en el cuerpo de la respuesta ni en almacenamiento accesible por JavaScript.*

- **Prioridad:** Alta · **Estado:** ☒ Implementado · **MoSCoW:** Must
- **Método de verificación:** Test
- **Origen:** `POST /api/auth/login` — `AuthController.java:99`
- **Verificación:** la respuesta deberá contener `Set-Cookie` con los tres atributos y no deberá contener el JWT en el cuerpo. Pruebas: `AuthServiceTest.loginConCredencialesCorrectas` , `AuthServiceTest.loginConContraseñaIncorrecta` .

RF-03 — Cierre de sesión con revocación efectiva *El sistema deberá permitir cerrar la sesión, y deberá invalidar el token emitido registrando su identificador (JTI) en una lista de revocación con tiempo de vida igual al tiempo restante del token, de modo que un token robado antes del cierre de sesión no siga siendo aceptado.*

- **Prioridad:** Alta · **Estado:**  Implementado · **MoSCoW:** Must
- **Método de verificación:** Test
- **Origen:** POST /api/auth/logout — AuthController.java:143 ; RedisBlacklistService.java
- **Verificación:** pruebas RedisBlacklistServiceTest.revocar_guarda_el_jti_con_el_ttl_restante , RedisBlacklistServiceTest.estaRevocado_true_si_existe_la_clave .
- **Nota:** resuelve OBS-07 y OBS-09 (Entrega 1B), donde se observó que la lista de revocación existía pero no estaba cableada a ningún endpoint.

RF-04 — Renovación de sesión *El sistema deberá permitir renovar una sesión vigente mediante un token de refresco, sin exigir que el usuario vuelva a introducir sus credenciales.*

- **Prioridad:** Media · **Estado:**  Implementado · **MoSCoW:** Should
- **Método de verificación:** Test
- **Origen:** POST /api/auth/refresh — AuthController.java:164
- **Verificación:** prueba JwtServiceTest.refresh_token_valido .

RF-05 — Consulta de la sesión activa *El sistema deberá permitir que el cliente consulte los datos de la sesión en curso (nombre de usuario, nombre completo y rol) a partir de la cookie de sesión, y deberá responder 401 cuando no exista sesión válida.*

- **Prioridad:** Alta · **Estado:**  Implementado · **MoSCoW:** Must
- **Método de verificación:** Demostración; Test
- **Origen:** GET /api/auth/me — AuthController.java:181
- **Verificación:** con sesión válida deberá responder 200 con {username, nombre, rol} ; sin sesión, 401 .

RF-06 — Limitación de intentos de autenticación *El sistema deberá bloquear temporalmente los intentos de autenticación de un mismo usuario tras 5 fallos consecutivos dentro de una ventana de 15 minutos, y el contador no deberá reiniciarse con cada nuevo fallo dentro de esa ventana.*

- **Prioridad:** Alta · **Estado:**  Implementado · **MoSCoW:** Must
- **Método de verificación:** Demostración; Test
- **Origen:** LoginAttemptService.java ; parámetros LOGIN_MAX_INTENTOS=5 , LOGIN_VENTANA_MINUTOS=15 (application.yml)
- **Verificación:** el sexto intento deberá responder 429 con cuerpo ProblemDetail . Pruebas: LoginAttemptServiceTest.bloqueada_al_alcanzar_el_limite , LoginAttemptServiceTest.fallo_subsiguiente_no_reinicia_ttl . Evidencia OWASP A07: docs/mediciones/sec/a07-rate-limit.txt .

RF-07 — Verificación de disponibilidad del servicio *El sistema deberá exponer un endpoint público de comprobación de disponibilidad que no requiera autenticación.*

- **Prioridad:** Baja · **Estado:**  Implementado · **MoSCoW:** Could
- **Método de verificación:** Test
- **Origen:** GET /api/auth/ping — AuthController.java:202
- **Verificación:** prueba AuthControllerTest.pingRespondePong .

RF-37 — Restablecimiento de contraseña por enlace *El sistema deberá permitir a un usuario restablecer su contraseña mediante un enlace de un solo uso, de vigencia limitada (30 minutos), enviado a su correo registrado, sin intervención del administrador, e invalidando las sesiones activas del usuario al completarse el restablecimiento.*

- **Prioridad:** Alta · **Estado:**  Implementado · **MoSCoW:** Must

- **Método de verificación:** Test; Demostración
- **Origen:** POST /api/auth/forgot , POST /api/auth/reset — AuthController.java ; PasswordResetService , PasswordResetTokenStore (token en Redis, sólo su SHA-256), SessionEpochService (época de invalidación).
- **Restricción de acceso:** ambos endpoints públicos; /forgot limitado a 3 solicitudes / 15 min por identificador y 10 / hora por IP (ResetRequestLimitService).
- **Verificación:** /forgot responde 202 idéntico exista o no la cuenta (no revela enumeración de usuarios); /reset responde 204 , o 400 si el token es inválido/expiró/ya se usó, o 422 si la contraseña incumple RNF-14. Pruebas: PasswordResetServiceTest (9 casos), PasswordResetTokenStoreTest , SessionEpochServiceTest , ResetRequestLimitServiceTest , AuthControllerTest ; frontend recuperar.component.spec.ts , restablecer.component.spec.ts .
- **Frontend:** pantallas /recuperar y /restablecer (frontend/src/app/auth/), con enlace desde el inicio de sesión.
- **Nota:** cierra el punto A22 de la revisión de requisitos contra ISO/IEC/IEEE 29148:2018. El correo destino no está verificado (limitación documentada en docs/etica/ETHICS.md).

3.2 Módulo de gestión de estudiantes

RF-08 — Listado paginado de estudiantes *El sistema deberá permitir consultar el listado de estudiantes de forma paginada, indicando en la respuesta el número de página, el tamaño, el total de elementos y el total de páginas.*

- **Prioridad:** Alta · **Estado:** ☒ Implementado · **MoSCoW:** Must
- **Método de verificación:** Test
- **Origen:** GET /api/estudiantes — academico/student/controller/StudentController.java
- **Acceso:** ADMINISTRADOR , ENTRENADOR , RECEPCIONISTA
- **Verificación:** pruebas StudentControllerTest.listar_devuelve_pagina , StudentServiceTest.listar_devuelve_pagina_envuelta .

RF-09 — Consulta de estudiante por identificador *El sistema deberá permitir consultar un estudiante por su identificador, y deberá responder 404 con cuerpo ProblemDetail cuando el identificador no corresponda a ningún registro.*

- **Prioridad:** Alta · **Estado:** ☒ Implementado · **MoSCoW:** Must
- **Método de verificación:** Test
- **Origen:** GET /api/estudiantes/{id} — academico/student/controller/StudentController.java
- **Verificación:** pruebas StudentControllerTest.buscarPorId_existente , StudentControllerTest.buscarPorId_inexistente_da_404 , StudentServiceTest.buscarPorId_inexistente_lanza_404 .

RF-10 — Registro de estudiante *El sistema deberá permitir que un usuario con rol ADMINISTRADOR registre un nuevo estudiante asociado a una persona, una categoría y un estado general existentes, con un código de estudiante único y fecha de ingreso, creando de forma transaccional el registro correspondiente.*

- **Prioridad:** Alta · **Estado:** ☒ Implementado · **MoSCoW:** Must
- **Método de verificación:** Test
- **Origen:** POST /api/estudiantes — academico/student/controller/StudentController.java
- **Acceso:** @PreAuthorize("hasAnyRole('ADMINISTRADOR', 'RECEPCIONISTA')") (esta nota decía solo ADMINISTRADOR; corregido 2026-08-12 para reflejar el código real — RECEPCIONISTA siempre

pudo registrar estudiantes).

- **Verificación:** deberá responder `201` con el recurso creado. Pruebas:
`StudentControllerTest.crear_devuelve_201` ,
`StudentServiceTest.crear_nuevo_estudiante_exito` .
- **Cambio respecto a la v1.0 de este documento:** el estudiante ya no se crea con nombre/apellido propios (esos viven en `Persona` , referenciada por `idPersona`); `EstudianteRequest` exige `idPersona` , `idCategoria` , `idEstadoGeneral` , `codigoEstudiante` y `fechaIngreso` , y admite opcionalmente `peso` y `altura` (ver hallazgo H-06 en `ETHICS.md`).
- **Frontend (2026-08-12):** se sirve desde la pantalla unificada `/personas` (`frontend/src/app/features/personas/`), que reemplaza a las antiguas `/estudiantes/registrar` y `/admin/crear-usuario` — ver `docs/superpowers/specs/2026-08-12-personas-unificado-design.md` .

RF-11 — Validación de la categoría del estudiante *El sistema deberá exigir que todo estudiante esté asociado a una categoría existente en el catálogo, mediante una clave foránea válida, y deberá responder `422 Unprocessable Entity` si la categoría indicada no existe o si falta.*

- **Prioridad:** Alta · **Estado:** ☒ Implementado — **contenido reescrito el 2026-07-30** · **MoSCoW:** Must
- **Método de verificación:** Test
- **Origen:** `EstudianteRequest.idCategoria` (`@NotNull`); `deportivo.categorias` como catálogo referenciado.
- **Verificación:** prueba `StudentControllerTest.crear_con_datos_invalidos_da_422` (pendiente de re-ejecutar contra el nuevo DTO — ver nota de cobertura en RNF-09).

Por qué cambió. La versión anterior de este requisito describía una validación de patrón de texto (`SUB-NN`) sobre un campo `VARCHAR` . Ese campo ya no existe: la categoría es ahora una entidad normalizada (`deportivo.categorias` , con `edad_min` / `edad_max`) referenciada por `idCategoria` . Se corrige el requisito para no describir una validación que el código ya no hace.

RF-11b — Registro de peso y altura del estudiante *El sistema deberá permitir registrar opcionalmente el peso y la altura de un estudiante al crearlo o actualizarlo, validando que sean valores positivos con hasta 3 dígitos enteros y 2 decimales, y tratándolos con la base legal y las condiciones declaradas más abajo.*

- **Prioridad:** Media · **Estado:** ☒ Implementado · **MoSCoW:** Should
- **Método de verificación:** Demostración; Inspección
- **Origen:** `StudentRequest.peso` , `.altura` (`@DecimalMin` , `@Digits` , opcionales); persistidos en `academico.estudiantes.peso/altura` (migración `V7`); devueltos en `StudentResponse` .
- **Decisión M7 (2026-09-08) — se conserva con base legal documentada:**
 - **Finalidad concreta:** seguimiento físico-deportivo del estudiante por el cuerpo técnico de la escuela (control del desarrollo físico apropiado a su categoría y edad, apoyo a la dosificación de carga en el entrenamiento). No se usan para ranking, selección ni decisiones automatizadas.
 - **Base legal:** consentimiento del representante legal conforme a la LOPDP (Ecuador, tratamiento de datos de niñas, niños y adolescentes), con **alcance específico "datos físico-deportivos"**, separado del consentimiento general de inscripción. Se registra por el mismo mecanismo de RF-39 — `POST /api/consentimientos` con `alcance = "DATOS_FISICO_DEPORTIVOS"` (constante `Consent.ALCANCE_DATOS_FISICO_DEPORTIVOS`), revocable y consultable por las mismas rutas — y queda sujeto a la compuerta de RF-51 para cualquier uso proactivo.
 - **Responsables del tratamiento:** la lectura de `peso` / `altura` se restringe a `ADMINISTRADOR` y `ENTRENADOR` — **aplicado el 2026-09-08:** `StudentController` omite ambos campos

(`StudentResponse.withoutPhysicalData()`) cuando el solicitante es `RECEPCIONISTA` , tanto en el listado como en el detalle. Prueba:

`StudentControllerTest.datos_fisicos_solo_para_administrador_y_entrenador` .

- **Conservación y supresión:** se conservan mientras el estudiante esté activo; la baja lógica los preserva por integridad del historial deportivo (RNF-22); se suprimen por RF-50 a solicitud del representante.
- El campo es y sigue siendo **opcional:** la ficha se puede crear y operar sin ellos.
- **Nota:** cierra el hallazgo **H-06** de `docs/etica/ETHICS.md` . Sus dos condiciones están cubiertas: (1) la restricción de lectura por rol está aplicada en `StudentController` (2026-09-08); (2) el consentimiento de alcance físico-deportivo tiene un valor de alcance propio y se registra por las rutas de RF-39 — es una acción del ADMINISTRADOR cuando el representante autoriza, no requiere lógica nueva (el envío proactivo sí queda bajo la compuerta de RF-51). El sistema no fuerza esa autorización como precondition de guardar el dato: peso/altura siguen siendo opcionales y su tratamiento se ampara en la base legal declarada; forzar la compuerta en el alta se deja como endurecimiento posterior, no como obligación de esta entrega.

RF-12 — Actualización de estudiante *El sistema deberá permitir que un usuario con rol ADMINISTRADOR actualice los datos propios de un estudiante existente (categoría, estado, código, fecha de ingreso, peso y altura).*

- **Prioridad:** Alta · **Estado:** ☒ Implementado · **MoSCoW:** Must
- **Método de verificación:** Test
- **Origen:** `PUT /api/estudiantes/{id}` — `academico/student/controller/StudentController.java`
- **Verificación:** prueba `StudentControllerTest.editar_actualiza_estudiante` .

RF-13 — Baja lógica de estudiante *El sistema deberá dar de baja a un estudiante marcándolo como inactivo, y no deberá eliminar físicamente el registro, con el fin de preservar el historial deportivo asociado.*

- **Prioridad:** Alta · **Estado:** ☒ Implementado · **MoSCoW:** Must
- **Método de verificación:** Demostración; Test
- **Origen:** `DELETE /api/estudiantes/{id}` — `academico/student/controller/StudentController.java`
- **Verificación:** deberá responder `204` y el registro deberá permanecer en la tabla con `activo = FALSE` . Pruebas: `StudentControllerTest.eliminar_devuelve_204` , `StudentServiceTest.eliminar_hace_baja_logica` .

RF-14 — Conteo de estudiantes activos por categoría *El sistema deberá informar el número de estudiantes activos de una categoría, identificada por su clave, y dicho conteo deberá calcularse en el motor de base de datos mediante un procedimiento almacenado versionado, no en la capa de aplicación.*

- **Prioridad:** Media · **Estado:** ☒ Implementado · **MoSCoW:** Should
- **Método de verificación:** Test
- **Origen:** `GET /api/estudiantes/conteo/categoria/{idCategoria}` — `academico/student/controller/StudentController.java` ; `academico.sp_contar_estudiantes_activos(p_categoria INT)`
- **Acceso:** `ADMINISTRADOR` , `ENTRENADOR`
- **Verificación:** pruebas `StudentControllerTest.contarActivos_delega_en_service` , `StudentServiceTest.conteo_por_categoria_delega_en_repositorio` .
- **Justificación:** cumple RD-02 (agregación obligatoriamente en el motor).
- **Cambio respecto a la v1.0:** la ruta y el parámetro cambiaron de `/conteo/{categoria}` (texto) a `/conteo/categoria/{idCategoria}` (entero); el procedimiento se movió del esquema `seguridad` a

academico y su parámetro de VARCHAR a INT .

RF-15 — Desactivación masiva por categoría *El sistema deberá permitir dar de baja lógica, en una sola operación transaccional, a todos los estudiantes activos de una categoría, e informar el número de registros afectados; dicha operación deberá ejecutarse mediante un procedimiento almacenado versionado.*

- **Prioridad:** Media · **Estado:** ☒ Implementado · **MoSCoW:** Should
 - **Método de verificación:** Test
 - **Origen:** POST /api/estudiantes/operaciones/desactivar-categoria —
academico/student/controller/StudentController.java ;
academico.sp_desactivar_estudiantes_categoria(p_categoria INT)
 - **Acceso:** @PreAuthorize("hasRole('ADMINISTRADOR')")
 - **Verificación:** prueba StudentControllerTest.desactivarCategoria_delega_en_service .
-

3.2b Módulo de catálogos y cuentas (nuevo en esta revisión)

Recursos con CRUD propio que aparecieron con la reestructuración de paquetes y no tenían requisito documentado hasta ahora.

RF-23 — Gestión del catálogo de categorías *El sistema deberá permitir crear, listar, consultar, actualizar y eliminar categorías deportivas, cada una definida por un nombre y un rango de edad (mínima y máxima).*

- **Prioridad:** Alta (bloquea RF-10/RF-11) · **Estado:** ☒ Implementado · **MoSCoW:** Must
 - **Método de verificación:** Test
 - **Origen:** CategoriaController (/api/categorias , 6 endpoints) —
deportivo/categoria/controller/CategoriaController.java
 - **Verificación:** CategoriaServiceTest (9 pruebas: paginación, alta, edición, baja lógica, validación de rango de edad), CategoriaControllerTest (7 pruebas: 200/201/204/400/404/422).
-

RF-24 — Gestión de cuentas de usuario como recurso propio *El sistema deberá permitir administrar cuentas de usuario (más allá del alta hecha en POST /api/auth/registro) de forma independiente.*

- **Prioridad:** Media · **Estado:** ☒ Implementado · **MoSCoW:** Should
- **Método de verificación:** Test
- **Origen:** UserAccountController (/api/usuarios , 5 endpoints) —
seguridad/user/controller/UserAccountController.java
- **Verificación:** UserAccountServiceTest (paginación, username duplicado, alta con contraseña codificada, alta con rol asignado, rol inexistente da error, persona inexistente, baja lógica, edición de rol/usuario/ contraseña, coherencia rol↔ficha), UserAccountControllerTest (200/201/204/400/422).
- **Cambio 2026-08-12:** UsuarioRequest agrega un campo rol opcional — si viene, se valida contra seguridad.roles y se asigna al crear (mismo criterio que AuthController.registro , pero sin forzar una Persona nueva). Permite darle acceso con cualquier rol a una Persona ya existente, no solo a las recién registradas. Frontend: pantalla unificada /personas (ver RF-10 y docs/superpowers/specs/2026-08-12-personas-unificado-design.md).
- **Cambio 2026-08-12 (edición y coherencia).** PUT /api/usuarios/{id} ahora sí aplica el cambio de rol (antes lo ignoraba) y la contraseña pasa a ser opcional: en blanco significa "no cambiarla". Además se valida la **coherencia rol↔ficha**: si la Persona tiene una ficha activa de Estudiante/Entrenador/Representante, su cuenta solo admite el rol correspondiente; sin fichas activas admite cualquiera (lo que permite crear la cuenta ENTRENADOR antes de la ficha). La guarda simétrica

vive en `StudentService.crear` . Ver `docs/superpowers/specs/2026-08-12-validaciones-rol-usuario-design.md` y `2026-08-12-coherencia-rol-y-vinculo-representante-design.md` .

RF-25 — Gestión de personas *El sistema deberá permitir administrar los datos personales base (nombre, cédula, correo, teléfono, fecha de nacimiento) independientemente del rol que la persona tenga en el sistema.*

- **Prioridad:** Media · **Estado:**  Implementado · **MoSCoW:** Should
- **Método de verificación:** Test
- **Origen:** `PersonController` (`/api/personas` , 6 endpoints)
- **Verificación:** `PersonServiceTest` (8 pruebas: paginación, búsqueda por cédula, unicidad de cédula/correo al crear y al editar, baja lógica), `PersonControllerTest` (6 pruebas: 200/201/204/400/422).
- **Frontend (2026-08-12):** `Persona` es la raíz de la que cuelgan `Usuario` / `Estudiante` / `Entrenador` / `Representante` ; la pantalla `/personas` refleja esa jerarquía en vez de crear la `Persona` por separado en cada flujo. `GuardianController` también se abrió a `RECEPCIONISTA` para crear/vincular representantes (editar/eliminar sigue exclusivo de `ADMINISTRADOR`) — ver spec de diseño.

RF-26 — Consulta de estados generales *El sistema deberá exponer el catálogo de estados generales utilizables por usuarios y estudiantes.*

- **Prioridad:** Baja · **Estado:**  Implementado (solo lectura — 1 endpoint) · **MoSCoW:** Could
- **Método de verificación:** Test
- **Origen:** `GeneralStatusController` — `seguridad/status/controller/GeneralStatusController.java`
- **Verificación:** `GeneralStatusServiceTest` (2 pruebas), `GeneralStatusControllerTest` (1 prueba).

3.3 Módulo deportivo

Actualizado 2026-07-30. RF-16 (entrenadores) pasó de  Modelado a  Implementado con la reestructuración de paquetes. RF-17 a RF-21 siguen con su **esquema de datos migrado y versionado** (`V3__dominio_deportivo.sql` , `V4__evaluaciones.sql`) pero **sin API REST propia todavía**. RF-22 (representantes) y el módulo de equipos no tienen ni esquema: son paquetes Java vacíos (ver nota al final de esta sección).

RF-16 — Gestión de entrenadores *El sistema deberá permitir registrar entrenadores asociados a una persona y a una cuenta de usuario, con especialidad, años de experiencia y certificación, garantizando que una misma persona o cuenta no pueda registrarse dos veces como entrenador.*

- **Prioridad:** Alta · **Estado:**  Implementado (cambió de Modelado) · **MoSCoW:** Must
- **Método de verificación:** Test
- **Origen:** `EntrenadorController` (`/api/entrenadores` , 5 endpoints) — `deportivo/entrenador/controller/EntrenadorController.java`
- **Esquema:** `deportivo.entrenadores` , con `UNIQUE` sobre `id_persona` e `id_usuario` (el vínculo con `Usuario` es nuevo respecto a la v1.0 de este documento).
- **Verificación:** `EntrenadorServiceTest` (7 pruebas: paginación con mapeo de persona/usuario, persona duplicada, usuario duplicado, alta válida, especialidad inexistente, baja lógica), `EntrenadorControllerTest` (5 pruebas: 200/201/204/404/422).

Actualizado 2026-08-12. `especialidad` pasó de texto libre a un catálogo (`deportivo.especialidades` , `FK id_especialidad` , nullable) — ver `EspecialidadController` / `EspecialidadService` y

`EspecialidadServiceTest` . El formulario de alta de entrenador en el frontend pasó de un input de texto a un `<select>` poblado desde `GET /api/especialidades/activas` .

RF-17 — Horarios recurrentes de entrenamiento El sistema deberá permitir definir horarios semanales recurrentes por categoría y entrenador, y deberá impedir que la hora de fin sea anterior o igual a la hora de inicio.

- **Prioridad:** Alta · **Estado:** ☒ Implementado · **MoSCoW:** Must — de él se generan las sesiones (RF-18).
- **Método de verificación:** Demostración
- **Origen:** `HorarioController` (`/api/horarios` , 4 endpoints) — `deportivo/horario/controller/HorarioController.java`
- **Verificación:** `HorarioServiceTest` , `HorarioControllerTest`

`POST/GET /api/horarios` , `DELETE /api/horarios/{id}` (baja lógica), todos `hasRole('ENTRENADOR')` y acotados al propio entrenador autenticado (404 si el horario no es suyo). Esquema: `deportivo.horarios_entrenamiento` , con `CHECK (hora_fin > hora_inicio)` y `CHECK (dia_semana BETWEEN 1 AND 7)` . De aquí se generan solas las sesiones del día que corresponde (ver RF-18): antes, cada sesión —fuera una recurrente o una extra— se creaba a mano.

RF-18 — Sesiones de entrenamiento El sistema deberá registrar cada sesión de entrenamiento con su fecha, categoría, entrenador responsable y estado, admitiendo únicamente los estados `PROGRAMADA`, `EN_CURSO`, `FINALIZADA` y `CANCELADA`.

- **Prioridad:** Alta · **Estado:** ☒ Implementado · **MoSCoW:** Must — de ella dependen asistencia (RF-19) e historial (RF-35).
- **Método de verificación:** Demostración
- **Origen:** `SesionEntrenamientoController` (`/api/sesiones` , 4 endpoints)
- **Verificación:** `SesionEntrenamientoServiceTest` , `SesionEntrenamientoControllerTest`

`POST /api/sesiones` (jornada extra, fuera del horario fijo), `GET /api/sesiones/hoy` y `/mias` . Estos dos últimos generan primero, de forma idempotente, la sesión de hoy de cada horario fijo activo que caiga en el día (`HorarioService.generarSesionesDeHoy()`) antes de listar — así ni el entrenador ni recepción dependen de que alguien cree la sesión a mano. Esquema: `deportivo.sesiones_entrenamiento` , con restricción `CHECK` sobre `estado` y FK opcional `id_horario` hacia el horario que la originó (null si es una jornada extra).

RF-19 — Registro de asistencia

Corrección (2026-09-07). El enunciado original exigía "el marcaje por RFID o manual" como si fueran una sola capacidad Must, cuando en realidad una vía está implementada y la otra no — un Must parcialmente cumplido no deja ver, sin leer la prosa, qué parte falta. Se divide en dos requisitos con estado independiente, siguiendo la misma disciplina que ya se aplicó en otras entradas de este documento.

RF-19a — Registro de asistencia por QR o manual El sistema deberá registrar la asistencia de cada estudiante a cada sesión mediante código QR (marcado por el propio estudiante) o lista manual (marcada por el entrenador), con estado `PRESENTE`, `TARDE`, `AUSENTE` o `JUSTIFICADO`, y deberá impedir que se registre más de una asistencia del mismo estudiante en la misma sesión.

- **Prioridad:** Alta · **Estado:** ☒ Implementado · **MoSCoW:** Must — precondition de notificaciones (RF-22) e historial (RF-35).
- **Método de verificación:** Demostración
- **Origen:** `AsistenciaQrController` (`POST /api/asistencias/qr/marcar`) · `AsistenciaSesionController.pasarLista` (`PUT /api/asistencias/sesion/{id}`)

- **Verificación:** `AsistenciaServiceTest.marcarPorQr_marca_presente_dentro_de_tolerancia`

RF-19b — Registro de asistencia por RFID *El sistema deberá admitir el marcaje de asistencia mediante lector RFID como vía adicional a RF-19a.*

- **Prioridad:** Baja · **Estado:**  Planificado · **MoSCoW:** Could — la escuela no dispone hoy de lector físico; sin ese hardware no hay forma de verificar la capacidad aunque se programe. El `CHECK` de `metodo` en el esquema ya admite el valor `'RFID'` (ver más abajo), así que activarla no exige migración, solo el lector y el endpoint.
- **Método de verificación:** Demostración
- **Origen:** endpoint RFID pendiente (sin controlador aún); el esquema `deportivo.asistencias` admite `metodo='RFID'`
- **Verificación:** Pendiente (requiere lector RFID físico) sin ese hardware no hay forma de verificar la capacidad aunque se programe. El `CHECK` de `metodo` en el esquema ya admite el valor `'RFID'` (ver más abajo), así que activarla no exige migración, solo el lector y el endpoint.

Esquema (común a ambos): `deportivo.asistencias`, con `UNIQUE (id_sesion, id_estudiante)` y `CHECK` sobre `metodo` y `estado`.

Dos vías de marcaje, y la distinción entre ambas se conserva en el dato:

Vía	Quién marca	metodo	hora_entrada
QR rotativo	el propio estudiante	QR	hora real de llegada, medida
Lista manual	el entrenador	MANUAL	vacía

La lista manual **no escribe una hora de llegada**. El entrenador afirma que el estudiante estuvo, no a qué hora entró; si pasa lista al terminar el entrenamiento, un `LocalTime.now()` guardaría la hora en que tecleó y quedaría escrito como si el chico hubiera llegado dos horas tarde a una sesión a la que llegó puntual. Así la columna significa algo preciso: si hay hora, la midió el QR; si no la hay, es palabra del entrenador.

RFID es RF-19b, arriba: sin lector físico, sin implementar.

Endpoints: `GET /api/asistencias/sesion/{id}` (nómina completa de la categoría, no solo quienes ya marcaron) y `PUT /api/asistencias/sesion/{id}` (upsert idempotente por estudiante). Una sesión con fecha posterior a hoy se puede consultar pero no editar: nadie pudo asistir todavía.

RF-20 — Evaluación diaria del desempeño *El sistema deberá permitir al entrenador evaluar a cada estudiante por criterios configurables (técnica, condición física, táctica y actitud), registrando la posición jugada ese día, y deberá impedir puntajes negativos y evaluaciones duplicadas del mismo estudiante y criterio dentro de una misma evaluación.*

- **Prioridad:** Media · **Estado:**  Implementado · **MoSCoW:** Should
- **Método de verificación:** Test
- **Origen:** `EvaluacionDiariaController` (`GET /api/evaluaciones/sesion/{idSesion}` , `PUT /api/evaluaciones/sesion/{idSesion}/jugadores` , `POST /api/evaluaciones/sesion/{idSesion}/finalizar`)
- **Verificación:** `EvaluacionDiariaServiceTest` , `EvaluacionDiariaControllerTest` .

Esquema: `deportivo.evaluaciones_diarias` , `deportivo.criterios_evaluacion` , `deportivo.detalle_evaluacion` , con `CHECK (puntaje >= 0)` y `UNIQUE (id_evaluacion, id_estudiante, id_criterio)` .

Corrección (2026-09-07). Esta entrada declaraba "solo esquema, sin endpoint todavía" — ya no es cierto: el controlador con sus 3 rutas existe desde antes de esta revisión. El estado no se había actualizado cuando se implementó. RF-21, en cambio, sigue correctamente en Modelado: existe la vista `deportivo.v_promedio_evaluacion` pero ningún controlador la expone.

RF-21 — Consulta de promedios de evaluación El sistema deberá calcular el promedio de puntajes por estudiante y evaluación en el motor de base de datos.

- **Prioridad:** Media · **Estado:** 🟡 Modelado · **MoSCoW:** Should — depende de RF-20, también solo esquema.
- **Método de verificación:** Inspección
- **Origen:** Vista `deportivo.v_promedio_evaluacion` ; sin controlador que la exponga
- **Verificación:** Inspección de esquema (vista `v_promedio_evaluacion` existe; sin endpoint)

Esquema: vista `deportivo.v_promedio_evaluacion` .

RF-33 — Agenda de partidos El sistema deberá permitir al entrenador agendar un partido de una categoría y registrar su resultado después de jugado.

- **Prioridad:** Alta · **Estado:** ✅ Implementado · **MoSCoW:** Must — base del dominio de partidos (RF-34).
- **Método de verificación:** Demostración
- **Origen:** `PartidoController` (`/api/partidos` , 4 endpoints) — `deportivo/partido/controller/PartidoController.java`
- **Verificación:** `PartidoServiceTest` , `PartidoControllerTest`

`deportivo.partidos` (GET-POST-PUT-DELETE `/api/partidos`). Los goles admiten nulo y no tienen valor por defecto: un partido recién agendado no va 0-0, todavía no se jugó, y `NULL` («sin resultado») y `0` («no metió ninguno») son cosas distintas. Un `CHECK` exige que estén los dos marcadores o ninguno: un marcador a medias no dice si se ganó. `GANADO/EMPATADO/PERDIDO/PENDIENTE` se calcula, no se almacena, para que no puedan contradecir al marcador.

Solo se lleva el marcador propio. El sistema es de una academia: «local/visitante» exigiría un catálogo de rivales que nadie va a mantener.

RF-34 — Convocatoria y once del partido El sistema deberá sugerir el once inicial de un partido a partir del rendimiento acumulado de las semanas previas, y deberá permitir al entrenador modificarlo y guardar la formación con la que efectivamente jugó.

- **Prioridad:** Alta · **Estado:** ✅ Implementado · **MoSCoW:** Must — regla de negocio central del módulo deportivo (tope de once titulares, exclusión por lesión).
- **Método de verificación:** Demostración
- **Origen:** `PartidoController` (GET/PUT/DELETE `/api/partidos/{id}/alineacion` , junto a la agenda de partidos)
- **Verificación:** `AlineacionServiceTest` , `PartidoControllerTest`

`deportivo.alineaciones` + `deportivo.alineacion_jugador` (GET-PUT-DELETE `/api/partidos/{id}/alineacion`).

La IA no elige a los jugadores. La regla es explícita y reproducible a mano: universo = plantel activo de la categoría; quedan fuera —con el motivo a la vista— el lesionado y quien no pisó un entrenamiento en la ventana; se ordena por promedio de evaluación de las últimas 4 semanas (`plantilla.semanas-rendimiento`), desempata por presencias y después por id para que dos llamadas con los mismos datos den lo mismo; y se titulariza al mejor de cada posición nominal, no a los once mejores promedios —eso podía

sugerir dos porteros y ningún defensa—. El modelo de lenguaje solo redacta un comentario sobre un once **ya decidido**, y solo cuando se le pide.

La ventana es lo que hace que la sugerencia se alimente semana a semana: el promedio histórico completo premia al que jugó bien hace un año por encima del que viene mejor ahora.

Sugerencia y decisión se guardan por separado: **si el entrenador guardó una alineación se devuelve esa; si no, la sugerida**. La sugerencia no se persiste sola —hacerlo convertiría una recomendación en un hecho histórico sin que nadie lo decidiera—.

Hasta V21 la alineación colgaba de la sesión de entrenamiento. V22 la movió al partido: decidir con quién se sale a jugar no es un hecho del entrenamiento, y atarla a la sesión obligaba a que solo pudieran alinearse los que fueron a **ese** entrenamiento.

RF-35 — Historial de asistencia de una sesión *El sistema deberá mostrar, para una sesión ya ocurrida, quiénes asistieron y quiénes no.*

- **Prioridad:** Media · **Estado:** ☒ Implementado · **MoSCoW:** Should — reporte sobre datos que ya existen por RF-19.
- **Método de verificación:** Demostración
- **Origen:** `SesionEntrenamientoController.historial` (GET /api/sesiones/{id}/historial)
- **Verificación:** `SesionEntrenamientoServiceTest.historialCuentaCadaEstadoPorSeparado`

GET /api/sesiones/{id}/historial . Se parte del **plantel** de la categoría y no de las filas de asistencia: si nadie pasó lista, la tabla está vacía y una consulta que solo lea de ahí diría «no había nadie convocado», que es distinto de «no se registró la asistencia de nadie». Por eso existe el estado `SIN_REGISTRO` , separado de `AUSENTE` .

RF-22 — Notificación a representantes *El sistema deberá notificar al representante legal cuando su representado marque asistencia o registre una lesión.*

- **Prioridad:** Media · **Estado:** ☒ Implementado · **MoSCoW:** Should — depende de RF-19/RF-31; hoy solo en-app, no correo/SMS/push (sección Trabajo futuro del
- **Método de verificación:** Demostración informe).
- **Origen:** `NotificationService` (creación automática al marcar asistencia/lesión);
`GuardianReportController` (GET /api/representante/notificaciones)
- **Verificación:** `NotificationServiceTest.con_consentimiento_se_notifica`

El rol REPRESENTANTE, el vínculo con sus representados y la tabla de consentimientos que este requisito exige como precondition (hallazgo H-04 de docs/etica/ETHICS.md) existen desde 2026-08-03. La notificación en sí es **en-app** (tabla `academico.notificaciones` , `NotificationService`); al marcar asistencia (`AsistenciaService.marcarPorQr`) o registrar una lesión (`LesionService.registrar`) se crea una fila por cada representante con vínculo activo, visible en GET /api/representante/notificaciones y marcabla como leída. No es correo/SMS/push — este proyecto no tiene infraestructura de envío externo, y agregarla requeriría credenciales que nadie tiene todavía; una notificación en-app satisface el requisito sin esa dependencia.

Actualización 2026-08-12 (asignación de representados). Hasta ahora el vínculo representante→estudiante solo existía en el backend: no había forma en la interfaz de ver ni asignar quién representa a un estudiante. La ficha de estudiante de /personas ahora lista los representantes vinculados y permite agregar y quitar, indicando la relacion del vínculo y cuál es el contacto_principal — que es justamente a quien apunta esta notificación. Ver docs/superpowers/specs/2026-08-12-coherencia-rol-y-vinculo-representante-design.md .

Corrección (2026-09-07). Esta nota decía que el paquete `academico.representante` estaba vacío (0 bytes) — ya no es cierto, y no lo era desde antes de esta revisión: quedó desactualizada cuando el paquete se llenó. Hoy (renombrado a `academico.guardian` en el rename al inglés de esta entrega) tiene DTOs, repositorios, servicios y entidades con contenido real, y tres controladores con rutas propias: `GuardianController` (`/api/representantes` , 9 rutas: CRUD, reactivación, vincular/desvincular estudiante), `GuardianReportController` (`/api/representante` , 7 rutas: informe del representado, comentario, notificaciones) y `ConsentController` (`/api/consentimientos` , 3 rutas: alta, revocación, consulta — ver A2 más abajo). Ver también RF-24/RF-25 para la gestión de representantes como recurso propio.

El módulo `deportivo.equipo` mencionado en la versión anterior de esta nota ya no existe ni siquiera como paquete vacío — se eliminó del código. No constituye un requisito de esta entrega: sin esquema ni endpoint, solo declarado.

Actualización 2026-08-12. Al reconciliar la base de Supabase para el módulo Inventario se descubrió que sí existía, creado a mano y fuera de control de versiones, un esquema de tablas para equipos, partidos y ejercicios (`deportivo.equipos` , `deportivo.partidos` , `deportivo.estadistica_partidos` , `deportivo.ejercicios` , `deportivo.entrenamiento_ejercicios`), todas vacías. Se versionó en `V16__equipos_partidos_ejercicios.sql` para que exista igual en cualquier entorno — es la mitad de base de datos de la limpieza de 2026-07-30 que en su momento solo tocó el código. Sigue sin `EquipoController` ni API REST: el esquema existe, la funcionalidad no. Estado: 🟡 solo esquema, no implementado. Este esquema no es un requisito especificado (no lleva id en la matriz); queda versionado como evidencia del hallazgo de 2026-08-12.

RF-31 — Registro y alta de lesiones El sistema deberá permitir al entrenador registrar una lesión de un estudiante (descripción y fecha estimada de retorno opcional), impidiendo una segunda lesión activa simultánea del mismo estudiante, y deberá permitir darla de alta cuando el estudiante se recupera.

- **Prioridad:** Alta · **Estado:** ✅ Implementado · **MoSCoW:** Must — condiciona la exclusión de lesionados en RF-34 y es dato de salud sensible (ver ETHICS.md).
- **Método de verificación:** Demostración

Esquema: `deportivo.lesiones` (`LesionController` , `LesionService`); el backend ya existía de una revisión anterior, pero sin frontend que lo consumiera. Se agregaron los botones "Marcar lesión" / "Dar de alta" a la pantalla de Evaluación diaria del entrenador, y se endureció `LesionController.registrar` : el `idEntrenador` de una cuenta ENTRENADOR ya no sale del cuerpo de la petición (que un entrenador podía manipular para registrar una lesión "a nombre de" otro), sino que se resuelve del token autenticado, mismo criterio que `SesionEntrenamientoController` .

RF-32 — Autoconsulta del estudiante sobre su equipo y desempeño El sistema deberá permitir a un estudiante autenticado consultar su propia categoría, su posición nominal, el entrenador de su próxima sesión programada, sus compañeros de equipo (solo nombre y posición, sin datos de contacto ni promedios — son menores de edad) y sus propias estadísticas de evaluación (promedio histórico por criterio, porcentaje de asistencia de los últimos 30 días e historial de lesiones propio).

- **Prioridad:** Media · **Estado:** ✅ Implementado · **MoSCoW:** Should — transparencia hacia el estudiante, no bloquea ningún otro requisito.
- **Método de verificación:** Demostración

`MyTeamController` (`GET /api/estudiante/mi-equipo` , `GET /api/estudiante/mi-informe`), solo rol ESTUDIANTE. Las estadísticas reutilizan tal cual la lógica que `InformeService` ya usaba para el informe que el representante ve de un representado (`construirInforme` , extraído como método común); el equipo es un endpoint nuevo (`MiEquipoService`) que cruza `academico.estudiantes` , `deportivo.categorias` ,

`deportivo.posiciones` y la sesión futura más próxima de `deportivo.sesiones_entrenamiento` para resolver el entrenador asignado.

3.4 Módulo de inventario (nuevo en esta revisión)

Cierra el schema `inventario` que `ADR-003` había reservado como diseño a futuro. Stock por cantidad agregada (no serializado por unidad individual): cada artículo tiene un `stock_actual` que los movimientos y las asignaciones ajustan, nunca por debajo de cero.

RF-27 — Gestión del catálogo de artículos de inventario El sistema deberá permitir crear, listar, consultar, actualizar y dar de baja artículos de inventario (uniformes, balones, implementos u otro), cada uno con un stock mínimo configurable para alertas de reposición.

- **Prioridad:** Alta (bloquea RF-28/RF-29) · **Estado:**  Implementado · **MoSCoW:** Must
 - **Método de verificación:** Test
 - **Origen:** `ItemController` (`/api/inventario/articulos` , 6 endpoints) — `inventario/item/controller/ItemController.java`
 - **Esquema:** `inventario.articulos` , con `CHECK` sobre `tipo` y `CHECK (stock_actual >= 0)` .
 - **Verificación:** `ItemServiceTest` (6 pruebas: alta con stock en cero, edición sin tocar el stock, baja lógica, paginación, stock bajo).
-

RF-28 — Registro de movimientos de stock El sistema deberá registrar entradas, salidas y ajustes de stock por artículo, y deberá impedir cualquier salida que deje el stock por debajo de cero.

- **Prioridad:** Alta · **Estado:**  Implementado · **MoSCoW:** Must
 - **Método de verificación:** Test
 - **Origen:** `StockMovementController` (`/api/inventario/movimientos` , 3 endpoints) — `inventario/movement/controller/StockMovementController.java`
 - **Esquema:** `inventario.movimientos_stock` , con `CHECK (cantidad > 0)` y `CHECK` sobre `tipo_movimiento` .
 - **Verificación:** `StockMovementServiceTest` (4 pruebas: entrada, ajuste y salida ajustan el stock correctamente; salida que dejaría el stock negativo se rechaza sin persistir nada).
-

RF-29 — Asignación y devolución de artículos El sistema deberá permitir asignar artículos a un estudiante o a un entrenador (nunca a ambos en la misma asignación), descontando el stock disponible, y deberá permitir marcar la devolución como `DEVUELTO` (repone el stock) o `PERDIDO` (no lo repone).

- **Prioridad:** Alta · **Estado:**  Implementado · **MoSCoW:** Must
 - **Método de verificación:** Test
 - **Origen:** `AssignmentController` (`/api/inventario/asignaciones` , 5 endpoints) — `inventario/assignment/controller/AssignmentController.java`
 - **Esquema:** `inventario.asignaciones` , con `CONSTRAINT chk_asignacion_destinatario` (exactamente un destinatario según `tipo_destinatario`) y `CHECK` sobre `estado` .
 - **Verificación:** `AssignmentServiceTest` (9 pruebas: asignación a estudiante y a entrenador, stock insuficiente, destinatario faltante, devolución que repone stock, pérdida que no lo repone, doble resolución, transición inválida a `ASIGNADO`, asignación inexistente).
-

RF-30 — Reporte de artículos con stock bajo El sistema deberá reportar, como cálculo agregado en el motor de base de datos, el total de artículos activos cuyo stock actual esté en o por debajo de su stock mínimo.

- **Prioridad:** Media · **Estado:**  Implementado · **MoSCoW:** Should

- **Método de verificación:** Test
- **Origen:** `GET /api/inventario/articulos/stock-bajo` — combina el listado (JPA) con el conteo agregado del procedimiento almacenado `inventario.sp_reporte_stock_bajo` (ver `docs/basedatos/CATALOGO-SP.md`).
- **Verificación:** `ItemServiceTest.stockBajo_combina_listado_y_total_del_procedimiento`.

3.5 Módulo administrativo, de pagos y de representantes (nuevo en esta revisión)

Cierra los puntos A1–A11 de la revisión contra 29148. El backend tiene controladores completos —con prueba— para pagos, consentimientos, informes al representante, auditoría, reportes y catálogos que ninguna línea del SRS describía. Cada uno se especifica aquí como requisito. Igual que en §3.2b, los recursos puramente administrativos (catálogos, gestión de representantes como recurso) se documentan directamente por haber aparecido como CRUD y no como una necesidad de actor articulada antes; las capacidades con implicación de dinero o de datos sensibles llevan su historia de usuario (HU-16, HU-17).

RF-38 — Gestión de pagos El sistema deberá permitir registrar el cobro de una membresía o de un pago diario a un estudiante, anular un pago dejando constancia de la anulación, y consultar los pagos de un estudiante y los ingresos agregados del mes y del histórico.

- **Prioridad:** Alta · **Estado:** ☒ Implementado · **MoSCoW:** Must
- **Método de verificación:** Test
- **Origen:** `PaymentController` (`/api/pagos` , 6 rutas: `POST /membresia` , `POST /diario` , `POST /{idPago}/anular` , `GET /estudiante/{idEstudiante}` , `GET /ingresos-mes` , `GET /ingresos-historico`).
- **Restricción de acceso:** `hasAnyRole('ADMINISTRADOR', 'RECEPCIONISTA')` en las 6 rutas. La anulación queda registrada por auditoría (`@Audited`).
- **Verificación:** `PaymentServiceTest` , `PaymentControllerTest` .
- **Esquema:** `academico.pagos` , con la anulación como cambio de estado, no como borrado (`V20__anulacion_de_pagos.sql`).

RF-39 — Registro y revocación del consentimiento del representante El sistema deberá permitir a un usuario ADMINISTRADOR registrar el consentimiento informado del representante legal de un estudiante, revocarlo dejando fecha de revocación, y consultar el estado de consentimiento de un estudiante.

- **Prioridad:** Alta · **Estado:** ☒ Implementado · **MoSCoW:** Must — es la precondition del hallazgo **H-04** de `docs/etica/ETHICS.md` y de las notificaciones a representantes (RF-22).
- **Método de verificación:** Test
- **Origen:** `ConsentController` (`/api/consentimientos` , 3 rutas: `alta` , `POST /{id}/revocar` , `GET /estudiante/{idEstudiante}`), `hasRole('ADMINISTRADOR')` .
- **Verificación:** `ConsentServiceTest` , `ConsentControllerTest` .

RF-40 — Informes y notificaciones al representante El sistema deberá permitir a un usuario REPRESENTANTE consultar la lista de sus representados, ver el informe de rendimiento de cada uno y comentar sobre ese informe, y listar y marcar como leídas sus notificaciones en la aplicación.

- **Prioridad:** Media · **Estado:** ☒ Implementado · **MoSCoW:** Should — extiende RF-22, que solo cubría la creación de la notificación.
- **Método de verificación:** Test; Demostración
- **Origen:** `GuardianReportController` (`/api/representante` , 6 rutas: `GET /estudiantes` , `GET /estudiantes/{id}/informe` , `POST /estudiantes/{id}/informe/comentario` , `GET`

/notificaciones , GET /notificaciones/no-leidas , POST /notificaciones/{id}/leida),
hasRole('REPRESENTANTE') .

- **Verificación:** GuardianReportControllerTest , NotificationServiceTest ,
StudentReportServiceTest .

RF-41 — Gestión de representantes como recurso *El sistema deberá permitir administrar los representantes legales (alta, consulta, edición, baja lógica, reactivación) y vincular o desvincular representantes de estudiantes indicando la relación y el contacto principal.*

- **Prioridad:** Media · **Estado:** ☒ Implementado · **MoSCoW:** Should
- **Método de verificación:** Test
- **Origen:** GuardianController (/api/representantes , 9 rutas). Alta, consulta y vinculación abiertas a ADMINISTRADOR y RECEPCIONISTA ; edición, baja, reactivación y desvinculación exclusivas de ADMINISTRADOR .
- **Verificación:** GuardianServiceTest , GuardianControllerTest .

RF-42 — Consulta de la bitácora de auditoría *El sistema deberá permitir a un usuario ADMINISTRADOR consultar de forma paginada los eventos de auditoría registrados.*

- **Prioridad:** Media · **Estado:** ☒ Implementado · **MoSCoW:** Should — RNF-08 obliga a **registrar** los eventos; poder consultarlos es una capacidad distinta, con su propio control de acceso.
- **Método de verificación:** Test
- **Origen:** GET /api/admin/auditorias — AuditController , hasRole('ADMINISTRADOR') .
- **Verificación:** AuditControllerTest , AuditServiceTest .

RF-43 — Reportes del sistema en PDF *El sistema deberá generar reportes en PDF de fichas de estudiantes, pagos, asistencias, evaluaciones y lesiones.*

- **Prioridad:** Media · **Estado:** ☒ Implementado · **MoSCoW:** Should
- **Método de verificación:** Test
- **Origen:** ReportController (/api/reportes , 5 rutas: /estudiantes-fichas , /pagos , /asistencias , /evaluaciones , /lesiones), cada una devuelve application/pdf .
- **Restricción de acceso:** fichas y pagos → ADMINISTRADOR , RECEPCIONISTA ; asistencias, evaluaciones y lesiones → ADMINISTRADOR , ENTRENADOR . Ningún rol sin relación con el dato puede generarlo.
- **Verificación:** ReportControllerTest , ReportServiceTest , ReportPdfServiceTest .
- **Nota:** estos reportes exportan datos de salud (lesiones, evaluaciones) y financieros (pagos) de menores; su control de acceso se declara aquí de forma explícita (antes no figuraba en el SRS).

RF-44 — Exportación de los datos propios *El sistema deberá permitir a cualquier usuario autenticado descargar en PDF los datos personales que el sistema guarda sobre él.*

- **Prioridad:** Media · **Estado:** ☒ Implementado · **MoSCoW:** Should — es, en la práctica, el derecho de acceso del titular sobre sus datos.
- **Método de verificación:** Test
- **Origen:** GET /api/usuarios/me/datos-pdf — PerfilController .
- **Verificación:** PerfilControllerTest .

RF-45 — Alertas del sistema *El sistema deberá presentar un panel de alertas operativas (estudiantes en riesgo, stock bajo y equivalentes) a los roles de gestión.*

- **Prioridad:** Baja · **Estado:** ☒ Implementado · **MoSCoW:** Could
- **Método de verificación:** Test

- **Origen:** `GET /api/alertas` — `AlertController` , `hasAnyRole('ADMINISTRADOR','RECEPCIONISTA')` .
 - **Verificación:** `AlertServiceTest` , `AlertControllerTest` .
-

RF-46 — Catálogos de especialidades y de posiciones *El sistema deberá permitir administrar el catálogo de especialidades de entrenador (CRUD, reservado a ADMINISTRADOR) y consultar el catálogo de posiciones activas.*

- **Prioridad:** Media (bloquea RF-16 y RF-34) · **Estado:** ☒ Implementado · **MoSCoW:** Should
 - **Método de verificación:** Test
 - **Origen:** `EspecialidadController` (`/api/especialidades` , 6 rutas; CRUD `hasRole('ADMINISTRADOR')` , `GET /activas` abierto a los tres roles de gestión) y `PosicionController` (`GET /api/posiciones/activas`).
 - **Verificación:** `EspecialidadServiceTest` , `EspecialidadControllerTest` , `PosicionControllerTest` .
-

RF-47 — Resumen y autoconsulta de asistencia *El sistema deberá permitir a los roles de gestión consultar el mapa de asistencia de una categoría, y a un estudiante autenticado consultar su propia asistencia.*

- **Prioridad:** Media · **Estado:** ☒ Implementado · **MoSCoW:** Should — reporte sobre datos que ya existen por RF-19a.
 - **Método de verificación:** Test
 - **Origen:** `GET /api/asistencias/mapa` — `ResumenAsistenciaController` (`ADMINISTRADOR` , `ENTRENADOR`); `GET /api/estudiante/mi-asistencia` — `MiAsistenciaController` (`ESTUDIANTE`).
 - **Verificación:** `ResumenAsistenciaControllerTest` , `MiAsistenciaControllerTest` .
-

RF-48 — Observaciones de texto libre del entrenador *El sistema permite registrar observaciones cualitativas en texto libre sobre un estudiante (`deportivo.observaciones_estudiante`).*

- **Prioridad:** No priorizado formalmente · **Estado:** ☒ Implementado · **MoSCoW:** Could — el bloqueo ético (H-02) quedó **resuelto por RNF-25** (tope de longitud a nivel de servidor y de motor, control de acceso y guía de redacción). Ya no está en Won't.
 - **Método de verificación:** Inspección
 - **Origen:** entidad `deportivo.observaciones_estudiante` (tabla + trigger de V4). No tiene aún controlador ni servicio JPA propio; el texto libre que el entrenador escribe hoy va por la descripción de lesión y la observación general de la evaluación, ambos cubiertos por RNF-25.
 - **Nota:** si en el futuro se le da un endpoint, hereda los controles de RNF-25 (el `CHECK` de longitud de motor ya está aplicado sobre esa tabla).
-

3.6 Cierre de hallazgos de protección de datos de menores (nuevo en esta revisión)

Responde al punto A2 de la revisión de septiembre. Cada hallazgo abierto de `docs/etica/ETHICS.md` se convierte aquí en un requisito con criterio verificable y condición de cierre, en vez de quedar solo como riesgo declarado. RNF-17 es el paraguas que los enlaza.

RF-49 — Cédula opcional y validada *El sistema deberá tratar la cédula (`seguridad.personas.cedula`) como dato opcional; cuando se proporcione, deberá validar el formato de cédula ecuatoriana (10 dígitos con dígito verificador) y rechazar con 422 los valores mal formados, y deberá garantizar su unicidad a nivel de esquema cuando haya valor.*

- **Prioridad:** Alta · **Estado:** ☒ Implementado (2026-09-08) · **MoSCoW:** Should

- **Método de verificación:** Test
- **Origen y controles:**
 - **Opcional:** `PersonRequest.cedula` y `RegisterRequest.cedula` ya no llevan `@NotBlank` ; `Person.cedula` es nullable .
 - **Dígito verificador:** anotación `@Cedula` (`common.validation.CedulaValidator`) — algoritmo módulo 10 para personas naturales (provincia 01–24 ó 30, tercer dígito 0–5, dígito verificador). Un valor ausente es válido.
 - **Unicidad cuando hay valor:** migración `V26__cedula_opcional_y_unica.sql` — `ALTER COLUMN cedula DROP NOT NULL + CREATE UNIQUE INDEX ... WHERE cedula IS NOT NULL` . `PersonService` y `AuthService` solo comprueban colisión de cédula si viene con valor.
- **Verificación:** `CedulaValidatorTest` (14 casos: opcional, 5 válidas, 8 inválidas); `PersonControllerTest` — `crear_sin_cedula_devuelve_201` , `crear_con_digito_verificador_invalido_da_422` , `crear_con_cedula_invalida_da_422` ; `PersonServiceTest.crear_sin_cedula_persiste` .
- **Criterio de cierre:** cumplido — `ETHICS.md` §H-01 marcado "corregido".
- **Nota sobre los datos de prueba/seed:** las cédulas ficticias de `db/seed.sql` (`0000000000` , `4000000001` , ...) se cargan por SQL directo y **no** pasan por la validación; siguen sirviendo como identificadores internos y son todas distintas (el índice único las admite). Las clases de prueba que crean una persona por API se ajustaron a cédulas válidas (`0912345675` , ...).
- **Fuera de alcance de este requisito:** el cifrado a nivel de columna, que se mantiene como recomendación para un despliegue con datos reales (H-01, RNF-21), no como obligación de esta entrega.

RF-50 — Supresión / anonimización de los datos del titular *El sistema deberá ofrecer a un usuario ADMINISTRADOR una operación que, ante una solicitud de supresión del representante legal, anonimice los datos identificativos de un estudiante (nombre, apellido, cédula, correo, teléfono, fecha de nacimiento y observaciones de texto libre) sustituyéndolos por valores neutros, conservando las claves foráneas y las estadísticas agregadas, y registrando el acto en la bitácora de auditoría.*

- **Prioridad:** Media · **Estado:** ☒ Implementado (2026-09-08) · **MoSCoW:** Should
- **Método de verificación:** Test; Demostración
- **Origen:** procedimiento almacenado versionado `academico.sp_anonimizar_estudiante(p_id_estudiante BIGINT)` (cumple RD-02, migración `V27__sp_anonimizar_estudiante.sql` , fuente documentada en `db/procs/sp_anonimizar_estudiante.sql`), invocado desde `StudentService.anonymize` vía `StudentRepository.anonymizeStudent` (`@Procedure`) y expuesto en `POST /api/estudiantes/{id}/anonimizar` — `@PreAuthorize("hasRole('ADMINISTRADOR')")` , `@Audited(accion = "ANONIMIZAR")` y `@CacheEvict` de la caché de listados. Cierra el hallazgo **H-03** e implementa el mecanismo de supresión que RNF-22 exige.
- **Qué hace el procedimiento:** sobre `seguridad.personas` deja `nombre = 'ANONIMIZADO'` , `apellido = 'ESTUDIANTE #<id>'` , `cedula = NULL` , `correo = 'anon+<id_persona>@anonimizado.local'` (neutro pero único, la columna es `NOT NULL UNIQUE`) , `telefono = NULL` , `foto = NULL` , `fecha_nacimiento = 1900-01-01` ; anonimiza y desactiva la cuenta de acceso si existe (`username = 'anon_<id>'` , `activo = FALSE`); reemplaza el texto libre escrito sobre el menor (`deportivo.observaciones_estudiante.texto` , `deportivo.lesiones.descripcion` , ambas `NOT NULL`) por un marcador; y da de baja lógica la ficha. No borra ninguna fila: las claves foráneas y los agregados de asistencia, evaluación y pagos quedan intactos.
- **Verificación:** `StudentServiceTest.anonimizar_delega_en_sp` y `anonimizar_estudiante_inexistente_lanza_404` ; `StudentControllerTest.anonimizar_devuelve_204` y

anonimizar_estudiante_inexistente_da_404 . Las pruebas unitarias corren sobre H2 sin el procedimiento (mismo criterio que el resto de SP del módulo, verificados con repo simulado); la comprobación a nivel de motor — V27 aplica limpio sobre db/schema.sql + V25 + V26 , campos identificativos neutros, texto libre suprimido, cuenta desactivada, ficha en baja lógica y **FKs/conteos de asistencia, evaluación y lesión intactos**, idempotente, excepción para id inexistente— se hizo sobre PostgreSQL 16 el 2026-09-08: transcripción en docs/mediciones/db/v27-anonimizacion.txt .

- **Condición de cierre:** cumplida — endpoint, procedimiento y pruebas en verde; ETHICS.md §3.4 y §H-03 actualizados; RNF-22 ya no dice que el mecanismo de supresión no exista.

RF-51 — Consentimiento vigente como precondition del envío de notificaciones *El sistema no deberá crear ni enviar una notificación al representante (RF-22 / RF-40) si no existe un consentimiento vigente de ese representante para el alcance correspondiente; la ausencia de consentimiento deberá registrarse como motivo de no-envío, no como error silencioso.*

- **Prioridad:** Alta · **Estado:** ☒ Implementado · **MoSCoW:** Must — condiciona RF-22.
- **Método de verificación:** Test
- **Origen:** NotificationService.crearParaCadaRepresentante consulta academico.consentimientos (ConsentRepository...RevocadoEnIsNull) por el alcance concreto — ALCANCE_NOTIFICACIONES_ASISTENCIA / _LESION , o el genérico ALCANCE_NOTIFICACIONES — antes de insertar; si no hay consentimiento vigente registra el motivo (log.info "no hay consentimiento vigente para ...") y no crea la fila. RF-39 cubre registrar y revocar el consentimiento. Cierra la mitad abierta de **H-04** (el envío proactivo del sistema); el documento de consentimiento del representante —**H-07**— vive en docs/etica/consentimiento/representante.md y mapea sus autorizaciones a estos mismos alcances.
- **Verificación:** NotificationServiceTest — con_consentimiento_se_notifica , sin_consentimiento_no_se_notifica (verifica never().save(...)), el_alcance_no_se_mezcla (el consentimiento de asistencia no habilita el de lesión). Como la comprobación consulta RevocadoEnIsNull en cada envío, al revocar dejan de crearse.
- **Criterio de cierre:** cumplido — ETHICS.md §H-04 pasa de "parcial" a "resuelto".

4. Requisitos no funcionales

Clasificados según las características de calidad de ISO/IEC 25010:2011. Los valores medidos provienen de la evidencia real versionada en docs/mediciones/ , no de estimaciones.

4.1 Eficiencia de desempeño

RNF-01 — Tiempo de respuesta *El sistema deberá responder a las consultas paginadas de estudiantes con un percentil 95 inferior a 200 ms con caché caliente e inferior a 500 ms con caché fría, bajo una carga de 50 usuarios virtuales concurrentes.*

- **Método de verificación:** Test
- **Verificación:** 5 corridas independientes de k6 (50 VUs, 30 s) por escenario (caché cálida y caché fría), reportando media, p90, p95 y p99.
- **Resultado medido (corrida de 2026-09-07):** caché cálida p95 promedio **19,94 ms** (IC 95 % ± 13,51), media 11,28 ms, throughput 386,09 RPS; caché fría p95 promedio **38,20 ms** (IC 95 % ± 2,01), media 17,74 ms, throughput 364,35 RPS; **0 % de errores** en ambas. El contraste cálida/fría es significativo (Mann-Whitney, \$p \approx 10^{-2483}\$). Ambos valores cumplen con amplio margen el umbral (< 200 ms cálida, < 500 ms fría). Evidencia: docs/mediciones/perf/REPORT.md y los k6-run*.json .

RNF-02 — Caché de consultas frecuentes *El sistema deberá cachear las consultas de listado de estudiantes con un tiempo de vida de 60 segundos, y la caché no deberá corromper la deserialización de tipos temporales.*

- **Método de verificación:** Test
- **Origen:** RedisCacheConfig.java ; CACHE_TTL_SECONDS=60 .
- **Nota:** se corrigió un defecto real por el cual la caché fallaba desde el segundo request (Jackson no leía el @class raíz) y otro por falta de soporte de java.time.Instant .

4.2 Seguridad

RNF-03 — Almacenamiento de contraseñas *El sistema no deberá almacenar contraseñas en texto plano ni de forma reversible; deberá utilizar BCrypt con factor de coste 12.*

- **Método de verificación:** Inspección

Verificación: db/seed.sql y SecurityConfig.java .

RNF-04 — Transporte cifrado *El sistema deberá ofrecer acceso mediante HTTPS con TLS 1.2 o superior.*

- **Método de verificación:** Análisis

Medido: TLS 1.3 vía nginx en :8443 . Evidencia: docs/mediciones/sec/a02-tls.txt (OWASP A02).

RNF-05 — Cabeceras de seguridad *El sistema deberá enviar en todas sus respuestas las cabeceras Content-Security-Policy , X-Content-Type-Options: nosniff , X-Frame-Options: DENY y, sobre HTTPS, Strict-Transport-Security .*

- **Método de verificación:** Análisis

Evidencia: docs/mediciones/sec/a05-cabeceras.txt (OWASP A05).

RNF-06 — Control de acceso por rol *El sistema deberá denegar toda petición a recursos protegidos que no presente una sesión válida (401) o cuyo rol no esté autorizado para la operación (403), y dicha verificación deberá aplicarse del lado del servidor con independencia de lo que muestre la interfaz.*

- **Método de verificación:** Test

Evidencia: docs/mediciones/sec/a01-acceso-roto.txt (OWASP A01).

RNF-07 — Ausencia de SQL dinámico *El sistema no deberá construir sentencias SQL mediante concatenación de cadenas; toda consulta deberá usar parámetros vinculados o procedimientos almacenados con parámetros nombrados.*

- **Método de verificación:** Análisis

Verificación: make audit incluye auditoría de SQL dinámico. Evidencia: docs/mediciones/sec/a03-inyeccion.txt (OWASP A03).

RNF-08 — Registro de auditoría de autenticación *El sistema deberá registrar cada intento de autenticación, exitoso o fallido, incluyendo marca de tiempo, dirección IP de origen e identificador del sujeto, sin registrar nunca la contraseña.*

- **Método de verificación:** Test

Evidencia: docs/mediciones/sec/a09-logging.txt (OWASP A09).

RNF-14 — Política de contraseñas *El sistema deberá exigir contraseñas de al menos 8 caracteres, con al menos una letra y un dígito, distintas del nombre de usuario, y de un máximo de 72 bytes; esta política deberá aplicarse de forma uniforme en el registro de usuarios, en la activación de acceso del estudiante, en el cambio administrativo de contraseña y en el restablecimiento por enlace (RF-37).*

- **Método de verificación:** Test; Inspección
- **Origen:** `PasswordPolicy.java` — única fuente de la regla; llamada por `AuthService.register` , `UserAccountService` , `StudentAccessService` y `PasswordResetService` . Incumplir la política devuelve `422 (ApiException)` .
- **Verificación:** `PasswordPolicyTest` (longitud, letra, dígito, igualdad con el usuario, límite de 72 bytes); `AuthServiceTest` , `UserAccountControllerTest` , `PasswordResetServiceTest` .
- **Nota:** cierra el punto A21 de la revisión contra 29148; sustituye el antiguo `@Size(min = 6)` de los DTO.

RNF-15 — Correo saliente de recuperación *El sistema deberá enviar el correo del enlace de restablecimiento (RF-37) mediante SMTP sobre STARTTLS a través de un proveedor autorizado (Gmail). Ante la indisponibilidad del proveedor, la solicitud de restablecimiento deberá seguir respondiendo de forma genérica y el fallo deberá quedar registrado. La configuración por defecto (`mail.enabled=false`) no envía correo y registra el enlace en la bitácora, de modo que el sistema funciona completo sin credenciales de correo (no rompe RNF-12).*

- **Método de verificación:** Test; Demostración
- **Origen:** `spring-boot-starter-mail` ; `spring.mail.*` y `mail:` en `application.yml` ; `SmtppasswordResetMailer` (`mail.enabled=true`)/ `LoggingPasswordResetMailer` (por defecto), seleccionados por `@ConditionalOnProperty` igual que los proveedores de IA.
- **Interfaz externa:** véase §4.5 (fila «SMTP saliente»).
- **Verificación:** `SmtppasswordResetMailerTest` (destinatario, asunto, enlace; un fallo del proveedor no se propaga), `LoggingPasswordResetMailerTest` .

RNF-16 — Frontera de datos hacia el proveedor de modelo de lenguaje *El sistema no deberá enviar al proveedor externo de modelo de lenguaje ningún dato que identifique a un menor. El texto generado (comentario de alineación y de evaluación) deberá construirse a partir de un perfil seudonimizado que contenga únicamente una referencia anónima, la categoría, la posición, los puntajes de evaluación, las asistencias del último mes y una bandera de lesión. Ante un fallo o una respuesta 503 del proveedor, la funcionalidad de dominio (guardar la evaluación, la alineación) deberá completarse igual y el comentario simplemente no aparecerá. Deberán declararse el proveedor y el modelo autorizados; la integración está deshabilitada por defecto (`IA_HABILITADO=false`).*

- **Método de verificación:** Test; Inspección
- **Origen:** paquete `common.ia` — `AnonymousPlayerProfile` (record con exactamente esos campos), `AIFeedbackGenerator` (interfaz), `GeminiFeedbackService` / `OpenAiFeedbackService` seleccionados por `ia.proveedor` .
- **Verificación:** `GeminiFeedbackServiceTest` , `OpenAiFeedbackServiceTest` , `PromptsFeedbackTest` (el prompt no contiene nombre, cédula ni correo); degradación segura ante 503 probada en esos mismos tests.
- **Nota:** hoy este diseño solo existe en el código; este RNF lo convierte en obligación — una refactorización futura que empiece a enviar nombres lo incumple. Cierra el punto A12 de la revisión.

RNF-17 — Protección de datos personales de menores *El sistema deberá tratar los datos personales de estudiantes menores según los principios declarados en `docs/etica/ETHICS.md` (minimización, limitación de la finalidad, confidencialidad), y deberá cerrar cada hallazgo abierto de ese documento mediante un requisito con criterio verificable, condición de cierre y fecha objetivo:*

Hallazgo	Requisito que lo cierra	Estado
H-01 — cédula en claro y sin validación	RF-49	✅ Resuelto (2026-09-08) — opcional + dígito verificador + índice único parcial

H-02 — texto libre sin control de contenido	RNF-25	✅ Resuelto (2026-09-08)
H-03 — sin mecanismo de supresión	RF-50 (implementa también RNF-22)	✅ Resuelto (2026-09-08) — SP <code>sp_anonimizar_estudiante</code> (v27) + endpoint <code>POST /api/estudiantes/{id}/anonimizar</code> auditado
H-04 / H-07 — consentimiento del representante	RF-39 (registro) + RF-51 (compuerta del envío) + plantilla <code>docs/etica/consentimiento/representante.md</code>	✅ H-04 (2026-09-08); ✅ H-07 (2026-09-09, plantilla del representante añadida)
H-05 — certificado TLS autofirmado	RNF-21	✅ Resuelto (2026-09-09) — despliegue en Render con certificado de <i>Google Trust Services</i> , HTTP→HTTPS y HSTS; autofirmado solo en el laboratorio
H-06 — peso y altura sin base legal	RF-11b — decisión M7 (2026-09-08): se conservan con finalidad y base legal documentadas; lectura restringida a ADMINISTRADOR/ENTRENADOR; consentimiento de alcance <code>DATOS_FISICO_DEPORTIVOS</code> por las rutas de RF-39	✅ Resuelto (2026-09-09) — ambas condiciones cerradas
H-08 — recursos sin <code>@PreAuthorize</code>	corregido 2026-07-30 (<code>a01-acceso-roto.txt</code>)	✅ Corregido
H-09 — correo de reseteo no verificado	limitación documentada de RF-37 (riesgo acotado, con mitigaciones: enlace de un solo uso, 30 min, respuesta genérica, rate limit, invalidación de sesiones). Fuera del alcance de la Entrega Final — el cierre pleno (doble opt-in en el alta) es una feature del tamaño de RF-37 y no figura en la revisión A2 del docente	🟪 Trabajo posterior a la entrega

- **Método de verificación:** Inspección
- **Origen:** `docs/etica/ETHICS.md` (inventario de datos y hallazgos), `docs/mediciones/sec/a01-acceso-roto.txt` (control de acceso por recurso).
- **Verificación:** revisión del cierre de cada hallazgo contra el criterio de su requisito; `a01-acceso-roto.txt` comprueba recurso por recurso, incluida la búsqueda por cédula.
- **Nota:** RF-11b y RF-48 dependen de este RNF. Cierra el punto A13 y responde al punto A2 de la revisión de septiembre (los hallazgos dejan de ser solo riesgos declarados y pasan a tener requisito que obliga a cerrarlos).

RNF-21 — Certificado y configuración TLS de producción *En producción el sistema deberá servirse sobre HTTPS con un certificado emitido por una autoridad reconocida (no autofirmado), con TLS 1.2 o superior, HSTS y redirección de*

HTTP a HTTPS. El estado actual —certificado autofirmado en el entorno de laboratorio— deberá declararse explícitamente (hallazgo H-05).

- **Método de verificación:** Demostración; Inspección
- **Origen:** `frontend/nginx.conf` , `docker-compose.yml` (:8443); `docs/mediciones/sec/a02-tls.txt` ; hallazgo H-05 de `ETHICS.md` .
- **Verificación:** `a02-tls.txt` reporta la versión de TLS negociada; inspección del emisor del certificado en el despliegue real.
- **Estado (2026-09-09):** ☒ cumplido en producción. El despliegue público en Render (`docs/despliegue/render.md`) sirve HTTPS con certificado emitido por *Google Trust Services* (autoridad reconocida), con redirección de HTTP a HTTPS y HSTS, verificado sobre `sged-frontend-jofa.onrender.com` y `sged-backend-5nh7.onrender.com` . El certificado autofirmado queda declarado como propio del entorno de laboratorio (`nginx` :8443), sin datos reales.
- **Nota:** complementa RNF-04, que solo exigía la versión del protocolo. Cierra el punto A17 y el hallazgo H-05.

RNF-22 — Conservación y supresión de datos *El sistema deberá declarar, por categoría de dato, el plazo de conservación y el procedimiento de supresión una vez cumplido ese plazo o atendida una solicitud del titular. La baja lógica —que preserva el historial— no deberá presentarse como un borrado: la supresión efectiva la realiza el procedimiento de anonimización de RF-50.*

- **Método de verificación:** Inspección
- **Origen:** `docs/etica/ETHICS.md` §3.4; `docs/despliegue/BACKUP.md` (retención de respaldos: 30 días); **RF-50** (mecanismo de supresión: `academico.sp_anonimizar_estudiante` , migración V27).
- **Verificación:** inspección de la tabla de plazos frente al inventario de datos de `ETHICS.md` ; el mecanismo de supresión se verifica por RF-50.
- **Nota:** cierra el punto A18. Desde el 2026-09-08 su mecanismo de supresión existe (RF-50), con lo que **H-03 queda resuelto**.

RNF-23 — Comportamiento ante indisponibilidad de Redis

Corrección (2026-09-08). *El enunciado original mezclaba en un solo RNF una capacidad (la autenticación falla cerrada) y otra (la caché de listados con `CacheErrorHandler`), de modo que no se podía saber, sin leer la nota, el estado de cada mitad. Se divide en dos requisitos con estado independiente, siguiendo la misma disciplina que RF-19a/RF-19b. **Ambas mitades quedan implementadas** (RNF-23a desde antes; RNF-23b el 2026-09-08). Responde al punto A4 de la revisión de septiembre y cierra el punto A19.*

RNF-23a — Autenticación falla-cerrado ante caída de Redis *Un token de acceso no podrá considerarse válido si no puede comprobarse contra la lista de revocación y la época de sesión; una caída de Redis deberá denegar el acceso a los recursos protegidos (401) en vez de aceptar tokens que podrían estar revocados.*

- **Estado:** ☒ Implementado
- **Método de verificación:** Análisis; Test
- **Origen:** `JwtAuthenticationFilter` (envuelve la comprobación en `try/catch` ; ante excepción no autentica y la petición continúa sin sesión), `RedisBlacklistService` , `SessionEpochService` .
- **Verificación:** análisis del flujo del filtro; `JwtAuthenticationFilterTest` ; prueba manual deteniendo el contenedor `sged_redis` (un recurso protegido responde 401 , no 500).

RNF-23b — Degradación de la caché de listados ante caída de Redis *Ante la indisponibilidad de Redis, la caché de listados (RNF-02) deberá degradarse a consulta directa a la base de datos mediante un `CacheErrorHandler` ; una caída de Redis no deberá producir 5xx en los endpoints de listado ni impedir la lectura.*

- **Estado:** ☒ Implementado (2026-09-08)
- **Método de verificación:** Test; Demostración

- **Origen:** `RedisCacheConfig` implementa `CachingConfigurer` y registra un `CacheErrorHandler` que, ante un fallo de Redis, registra el incidente en `WARN` y **no relanza** en los cuatro casos (`get/put/evict/clear`); Spring entonces ejecuta el método anotado —que consulta la base—.
- **Verificación:** `RedisCacheErrorHandlerTest` (los cuatro `handle*Error` no propagan la excepción); prueba manual deteniendo el contenedor `sged_redis` y comprobando que `GET /api/estudiantes` sigue respondiendo `200` con datos.
- **Criterio verificable:** el `CacheErrorHandler` está registrado y la prueba pasa en verde. **Cumplido.**

4.3 Fiabilidad y mantenibilidad

RNF-09 — Cobertura de pruebas *El sistema deberá mantener una cobertura de líneas y de ramas (branches) igual o superior al 70 %, verificada automáticamente en la construcción.*

- **Método de verificación:** Análisis

Corrección (2026-09-07). El enunciado y la cifra de abajo citaban 60 % de instrucciones — ese nunca fue el valor configurado en `pom.xml` (que exige 70 % en `LINE` y en `BRANCH`, sin excepciones de paquete) y la cifra estaba fechada 2026-07-30, mucho antes del estado actual del código. Cifra vigente, regenerada el 2026-09-07 tras el rename de identificadores de esta entrega (`./mvnw clean verify`): **84,63 % de líneas (2638/3117) y 71,24 % de branches (664/932), 550 pruebas en 74 clases, 200 clases analizadas — CUMPLE el 70 % en ambas métricas.** Desglose por subdominio en `docs/informe/main.tex` (Tabla `tab:cobertura-por-paquete`, 25 filas) y dato crudo en `docs/mediciones/jacoco/jacoco.csv`. La bitácora original de esta jornada (72,5 % el 2026-07-30) se conserva abajo sin alterar, como registro histórico de cómo se llegó hasta acá — no como cifra vigente.

- **Medido el 2026-07-30 con construcción limpia** (`./mvnw clean test`): **72,5 % (2507 instrucciones cubiertas de 3457) — cifra histórica, no vigente (ver corrección arriba).**
- 102 pruebas en 17 clases, **todas pasan** (0 fallos, 0 errores).
- **Por qué "construcción limpia" aparece explícito aquí:** la primera medición de esta jornada se hizo con `./mvnw test` sobre un `target/` que aún conservaba `.class` de antes de la reestructuración de paquetes. El reporte archivado llegó a listar paquetes que ya no existen en el código fuente (`org.uteq.backend.auth.*`, `org.uteq.backend.estudiante.*`). La cifra publicada ahora proviene de `clean test`, y el reporte de `docs/mediciones/jacoco/` contiene solo los 27 paquetes reales.
- **Historial de esta cifra en la misma jornada**, para que quede trazable: primero se detectó una regresión real a 39,8 % (los 5 recursos nuevos de la reestructuración — Categoría, Entrenador, Usuario, Persona, EstadoGeneral — no tenían ninguna prueba propia); se agregaron 57 pruebas nuevas (`CategoriaServiceTest`, `CategoriaControllerTest`, `EntrenadorServiceTest`, `EntrenadorControllerTest`, `UserAccountServiceTest`, `UserAccountControllerTest`, `PersonServiceTest`, `PersonControllerTest`, `GeneralStatusServiceTest`, `GeneralStatusControllerTest`) y la cobertura subió a 72,5 %.
- Evidencia: `docs/mediciones/jacoco/` (reporte regenerado con la ejecución que incluye las 101 pruebas).

RNF-10 — Tipificación de errores *El sistema deberá responder los errores en formato `ProblemDetail` (RFC 9457), con `type`, `title`, `status`, `detail` e `instance`, y no deberá exponer trazas de pila ni detalles internos de implementación.*

- **Método de verificación:** Test

Origen: `GlobalExceptionHandler.java`, `ProblemDetailsAuthHandlers.java`.

RNF-11 — Versionado del esquema de datos *Todo cambio en el esquema de base de datos deberá aplicarse mediante una migración Flyway versionada e incremental; no deberá modificarse el esquema de forma manual ni automática por el ORM en tiempo de arranque.*

- **Método de verificación:** Inspección

Origen: V1 a V6 ; ddl-auto: validate .

RNF-20 — Calidad estática del código *La construcción deberá superar el quality gate de SonarQube configurado para el proyecto; una condición incumplida deberá fallar la construcción en CI.*

- **Método de verificación:** Test; Inspección
- **Origen:** docs/mediciones/sonarqube/ (quality-gate.json , measures.json , issues.json); paso de análisis en .github/workflows/ .
- **Verificación:** quality-gate.json → "status":"OK" , "caycStatus":"compliant" .
- **Nota:** complementa RNF-09 (cobertura) y el análisis de SpotBugs. Cierra el punto A16.

RNF-24 — Respaldo y recuperación de la base de datos *El sistema deberá contar con: (a) un respaldo diario de la base de datos completa (pg_dump -F c), automatizado, con retención mínima de 30 días y destino en un almacenamiento privado externo al repositorio y —cuando el plan contratado lo permita— externo también al proveedor de base de datos; (b) la recuperación punto-en-el-tiempo (PITR) del proveedor gestionado (Supabase) declarada explícitamente con su ventana de retención real según el plan vigente; (c) un objetivo de punto de recuperación (RPO) igual o inferior a 24 horas y un objetivo de tiempo de recuperación (RTO) medido, no estimado; (d) un procedimiento de restauración documentado y verificado con evidencia archivada y fechada de al menos una ejecución real contra una base separada.*

- **Método de verificación:** Demostración; Inspección
- **Origen:** docs/despliegue/BACKUP.md (frecuencia, retención, destino, procedimiento pg_dump -F c / pg_restore), docs/despliegue/RUNBOOK.md §5 (restauración).
- **Criterio verificable:** BACKUP.md fija el destino de almacenamiento, el RPO y la retención del PITR del proveedor; existe docs/mediciones/backup/restauracion-AAAA-MM-DD.md con el log de una restauración real y el tiempo cronometrado; el RTO y el RPO figuran en la matriz de trazabilidad con fecha.
- **Condición de cierre:** el archivo de evidencia de restauración existe y el RTO consta como medido (no "pendiente de medir"); BACKUP.md deja de decir que el destino "todavía no está fijado".
- **Estado (2026-09-09):** ☒ cumplido. (a) destino fijado (almacenamiento privado del equipo, fuera del repo; pg_dump -F c diario acotado a los 4 esquemas de la app); (b) **PITR declarado:** el plan **Free de Supabase no ofrece PITR ni respaldos gestionados**, por lo que el pg_dump diario es la única copia y el punto de recuperación es siempre el de ese dump; (c) **RPO ≤ 24 h** (cadencia del pg_dump) y **RTO medido** — pg_restore ≈ 1 s, recuperación completa 3–5 min; (d) **evidencia archivada y fechada:** [docs/mediciones/backup/restauracion-2026-09-09.md](#) — restauración real del respaldo de producción contra un postgres:17 separado, 0 errores, con verificación de esquema, los 12 procedimientos almacenados, conteos de filas y un flujo de lectura (login del ADMINISTRADOR
 - listado por sp_contar_estudiantes_activos).
- **Nota:** cierra el punto A20 y responde al punto A3 de la revisión de septiembre (RPO explícito, PITR declarado, destino fijado, evidencia de restauración archivada).

RNF-25 — Control de contenido de las observaciones de texto libre *Todo texto libre que se escriba sobre un estudiante menor —la observación general de la evaluación diaria, la descripción de lesión y deportivo.observaciones_estudiante.texto — deberá tener un límite de longitud aplicado en el servidor y a nivel de motor, y su lectura deberá estar restringida a los roles del cuerpo técnico y de coordinación (ADMINISTRADOR / ENTRENADOR), nunca a RECEPCIONISTA, REPRESENTANTE ni ESTUDIANTE.*

- **Estado:** ☒ Implementado (2026-09-08)
- **Método de verificación:** Inspección; Test
- **Origen y controles:**

- **Límite de longitud en el servidor:** `@Size(max = 1000)` en la descripción de lesión (`RegistrarLesionRequest`); `@Size(max = 255)` en la observación de asistencia (`PasarListaDtos.MarcaAsistencia`); guarda de 2000 caracteres en `EvaluacionDiariaService.finalizar` para la observación general.
- **Límite a nivel de motor** (defensa en profundidad y cobertura de `observaciones_estudiante`, que aún no tiene código JPA): migración `V25_limite_texto_libre_menores.sql` con `CHECK (char_length(...) <= 2000)` en `deportivo.evaluaciones_diarias.observacion_general` y `deportivo.observaciones_estudiante.texto`, y `<= 1000` en `deportivo.lesiones.descripcion` (la observación de asistencia ya es `VARCHAR(255)`).
- **Control de acceso:** `EvaluacionDiariaController` y las vías de asistencia están `@PreAuthorize` a `ADMINISTRADOR / ENTRENADOR`; `LesionController` a `ENTRENADOR`. Ningún otro rol accede al texto.
- **Guía de redacción en la interfaz:** el punto de captura de texto libre que existe hoy —el formulario de lesión de la pantalla de evaluación diaria— muestra una guía (`evaluacion-diaria.component`): "anotó solo lo relacionado con la lesión, evitá juicios de valor, datos de salud no verificados y comentarios sobre terceros", con contador de caracteres y `maxlength`.
- **Verificación:** `EvaluacionDiariaServiceTest.observacionGeneralConTopeDeLongitud` (rechaza 2001 caracteres); `LesionControllerTest` cubre el `@Size` de la descripción; `evaluacion-diaria.component.spec` (RNF-25: ... descripción demasiado larga no llama al backend); inspección de los `@PreAuthorize`.
- **Nota:** responde al punto A2 de la revisión de septiembre; cierra **H-02**.

4.4 Portabilidad

RNF-12 — Reproducibilidad en un solo comando *El sistema deberá levantarse completo (base de datos, caché, backend y frontend) desde una clonación limpia del repositorio mediante un único comando, en menos de dos minutos y sin configuración manual adicional más allá de copiar el archivo de variables de entorno de ejemplo.*

- **Método de verificación:** Demostración

Origen: `make up`. Verificado mediante clonación real independiente en carpeta separada, no solo reiniciando el volumen local.

RNF-13 — Fijación de dependencias de infraestructura *Las imágenes de contenedor de base de datos y caché deberán fijarse por digest SHA-256 y no por etiqueta móvil, para garantizar que dos construcciones del mismo commit usen exactamente los mismos binarios.*

- **Método de verificación:** Inspección

Origen: `docker-compose.yml` (digests reales aplicados por `scripts/pin-digests.sh`).

4.5 Interfaces externas

Interfaz	Descripción	Protocolo / Formato	Responsable	Referen
API REST SGED	Interfaz principal de la aplicación (frontend ↔ backend)	HTTPS + JSON (RFC 8259), OpenAPI 3.0	<code>backend/src/main/java/.../controller/</code>	Swagger UI <code>ui.html</code>

Proveedor IA (LLM)	Generación de comentarios de alineación y reportes	HTTPS + JSON, proveedor configurable	deportivo.ia.service.AiCommentaryService	paquete cc 16)
Terminación TLS (nginx)	Descarga SSL/TLS, proxy reverso, rate-limit	TLS 1.2/1.3, HTTP/1.1, HSTS, CSP	frontend/nginx.conf, docker-compose.yml	Puerto externo 80
Base de datos PostgreSQL	Persistencia transaccional y vistas	PostgreSQL 16, pgjdbc	Flyway migrations db/migration/	Esquemas: academico, c inventario
Caché Redis 7	Sesiones, revocación JWT, listas de acceso, token de reseteo	Redis RESP3, TTL configurable	RedisBlacklistService, PasswordResetTokenStore, SessionEpochService	redis://red
SMTP saliente	Correo del enlace de restablecimiento de contraseña (RF-37 / RNF-15)	SMTP + STARTTLS (:587), cuerpo HTML	Smtppasswordresetmailer (mail.enabled=true)	Gmail smtp defecto nc (LoggingPas
Seed de datos	Datos base (roles, categorías, estados)	SQL idempotente	db/seed.sql	Roles: ADM ENTRENAD RECEPCION REPRESENT ESTUDIANTE

4.6 Máquinas de estado del dominio

Entidad	Estados	Transiciones válidas	
deportivo.sesiones_entrenamiento.estado	PROGRAMADA → EN_CURSO → FINALIZADA / CANCELADA	PROGRAMADA→EN_CURSO (inicio real), EN_CURSO→FINALIZADA (cierre), PROGRAMADA/EN_CURSO→CANCELADA (anulación)	SesionEntre (las crean e anulación : cierre del fl EvaluacionD
deportivo.asistencias.estado	PRESENTE, TARDE, AUSENTE, JUSTIFICADO, SIN_REGISTRO	SIN_REGISTRO→cualquier otro (upsert idempotente); PRESENTE/TARDE/JUSTIFICADO ↔ AUSENTE (corrección entrenador)	AsistenciaQ /api/asiste AsistenciaS (PUT /api/as
deportivo.partidos.estado (calculado)	PENDIENTE (sin	Automático según marcador_local / marcador_visitante al PUT	PartidoCont

	marcador) / GANADO / EMPATADO / PERDIDO		
deportivo.alineaciones.estado	SUGERIDA / CONFIRMADA	SUGERIDA→CONFIRMADA (entrenador guarda)	PartidoCont /api/partid
academico.estudiantes.activo	TRUE (activo) / FALSE (baja lógica)	TRUE→FALSE (DELETE lógico); FALSE→TRUE (reactivación admin)	StudentCont /api/estudi StudentCont /api/estudi
seguridad.usuarios.activo	TRUE / FALSE	TRUE→FALSE (baja), FALSE→TRUE (reactivar)	UserAccount
seguridad.tokens_revocados (JTI)	VIGENTE / REVOCADO (TTL = resto de vida del token)	VIGENTE→REVOCADO (logout)	AuthControl

4.7 Matriz de permisos por rol y operación

Controlador / Recurso	Endpoint	ADMINISTRADOR	ENTRENAD
AuthController	POST /api/auth/registro	✓	
AuthController	POST /api/auth/login	✓	✓
AuthController	POST /api/auth/logout	✓	✓
AuthController	POST /api/auth/refresh	✓	✓
AuthController	GET /api/auth/me	✓	✓
AuthController	GET /api/auth/ping	Público	Público
StudentController	GET /api/estudiantes	✓	✓
StudentController	GET /api/estudiantes/{id}	✓	✓
StudentController	POST /api/estudiantes	✓	
StudentController	PUT /api/estudiantes/{id}	✓	
StudentController	DELETE /api/estudiantes/{id}	✓	
StudentController	GET /api/estudiantes/conteo/categoria/{id}	✓	✓

StudentController	POST /api/estudiantes/operaciones/desactivar-categoria	✓	
PersonController	GET/POST/PUT/DELETE /api/personas	✓	
UserAccountController	GET/POST/PUT/DELETE /api/usuarios	✓	
GeneralStatusController	GET /api/estados-generales	✓	✓
CategoriaController	GET/POST/PUT/DELETE /api/categorias	✓	✓
EntrenadorController	GET/POST/PUT/DELETE /api/entrenadores	✓	✓
HorarioController	GET/POST/DELETE /api/horarios		✓ (propia
SesionEntrenamientoController	GET/POST /api/sesiones	✓	✓ (propias/hc
SesionEntrenamientoController	GET /api/sesiones/{id}/historial	✓	✓
AsistenciaSesionController	PUT /api/asistencias/sesion/{id} (QR/manual)	✓	✓
AsistenciaSesionController	GET /api/asistencias/sesion/{id}	✓	✓
EvaluacionDiariaController	GET/PUT/POST /api/evaluaciones/sesion/{id}		✓ (propia
PartidoController	GET/POST/PUT/DELETE /api/partidos	✓	✓ (propia
PartidoController	GET/PUT/DELETE /api/partidos/{id}/alineacion	✓	✓ (propia
LesionController	POST /api/lesiones		✓ (propia
GuardianController	GET/POST/PUT/DELETE /api/representantes	✓	
GuardianReportController	GET /api/representante/notificaciones		
ConsentController	POST/DELETE /api/consentimientos	✓	
ItemController	GET/POST/PUT/DELETE /api/inventario/articulos	✓	
ItemController	GET /api/inventario/articulos/stock-bajo	✓	
StockMovementController	GET/POST /api/inventario/movimientos	✓	
AssignmentController	GET/POST/PUT /api/inventario/asignaciones	✓	

Nota: "propias" = recurso propiedad del usuario autenticado (p. ej., horarios/sesiones/partidos del entrenador logueado). El control de acceso se aplica vía `@PreAuthorize` y filtros en service. Ver `SecurityConfig` y cada `@Controller`.

4.8 Correspondencia con ISO/IEC/IEEE 29148:2018 (Anexo C)

Cláusula 29148:2018	Título	Sección SRS equivalente	Comentario
5.1	Propósito	1.1	Alcance del sistema SGED
5.2	Alcance	1.2	Contexto escuela de fútbol formativo
5.3	Definiciones y abreviaturas	1.4	Glosario de términos técnicos
5.4	Referencias	1.5	Normas, ADRs, specs vinculados
5.5	Visión general del producto	2.1	Arquitectura 3 capas, esquemas BD
5.6	Necesidades de los interesados	2.2	5 actores, roles técnicos
5.7	Restricciones	2.3	Tecnológicas, legales, éticas
5.8	Suposiciones y dependencias	2.3	Infraestructura, proveedores, hardware
5.9	Requisitos funcionales	3.1–3.6	RF-01 a RF-51 organizados por módulo (§3.6: cierre de hallazgos de datos de menores)
5.10	Requisitos de calidad (no funcionales)	4.1–4.9	RNF-01 a RNF-25 por ISO 25010 (incluye usabilidad y accesibilidad en §4.9)
5.11	Requisitos de interfaz	4.5	API REST, IA, TLS, BD, Redis, Seed
5.12	Requisitos de verificación	4.6, 4.7	Máquinas de estado, matriz permisos
5.13	Trazabilidad	5	Matriz CSV, bitácora observaciones
5.14	Gestión de cambios	6	Historial de versiones del documento
5.15	Aprobación	7	Firmas y registro de entregas

Esta tabla permite la auditoría de cumplimiento de la norma sin reordenar la estructura del documento. Cada cláusula 29148 se mapea a la sección SRS que la cubre.

4.9 Usabilidad y accesibilidad

RNF-18 — Usabilidad medible (SUS) *El sistema deberá alcanzar una media igual o superior a 68 en la escala System Usability Scale (SUS), medida sobre una muestra de al menos 10 participantes externos al equipo de desarrollo.*

- **Método de verificación:** Análisis (encuesta estructurada)
- **Origen:** docs/mediciones/sus/ — INSTRUMENTO-SUS.md , respuestas.csv , REPORT.md (regenerado por scripts/sus-analysis.py), INTERPRETACION.md .
- **Resultado medido (2026-08-18, n = 15):** media SUS **69,33** (IC 95 % 58,87–79,79 con t de Student; DT 18,89; mediana 70,00), grado C («Aceptable»). La estimación puntual cruza el umbral por 1,33 puntos;

el intervalo de confianza todavía incluye valores por debajo de 68, de modo que la afirmación defendible es "la mejor estimación está por encima del umbral", no "el sistema lo supera con holgura".

- **Nota:** cierra el punto A14 — el umbral existía en la medición pero no en la especificación.

RNF-19 — Accesibilidad Las páginas del frontend deberán obtener una puntuación de accesibilidad igual o superior a 90 en Lighthouse, tomando WCAG 2.1 nivel AA como marco de referencia.

- **Método de verificación:** Test (auditoría Lighthouse)
- **Origen:** docs/mediciones/lighthouse/ — REPORT.md y los *.report.json de escritorio y móvil.
- **Resultado medido:** accesibilidad **100/100** en las tres corridas, tanto en escritorio como en móvil (umbral ≥ 90). SEO 63 y rendimiento por debajo del umbral se documentan aparte en ese informe y no forman parte de este RNF.
- **Nota:** cierra el punto A15; especialmente relevante porque la aplicación la usan representantes y estudiantes menores de edad.

5. Trazabilidad

La correspondencia entre cada requisito, su implementación, su prueba automatizada y su evidencia empírica se mantiene en [docs/trazabilidad/matriz.csv](#).

El seguimiento de las observaciones emitidas por el docente en las entregas previas se mantiene en [docs/observaciones/](#).

6. Control de cambios del documento

Versión	Fecha	Entrega	Cambios principales
1.0	2026-05-xx	Entrega 1A	Versión inicial.
1.1	2026-07-15	Entrega 1B / Tercera	Resuelve OBS-01 y OBS-12; se añaden módulos de catálogos, inventario y dominio deportivo.
1.2	2026-08-24	Entrega Final (v1.0.0)	Reestructuración de paquetes academico/deportivo/seguridad; RF-35 e historial de asistencia; cierre de trazabilidad (matriz de 47 filas).
1.3	2026-09-04	Entrega Final (v1.0.0)	Campo MoSCoW explícito en los 36 RF (11 no tenían prioridad formal); matriz de trazabilidad ampliada a 50 filas.
1.4	2026-09-07	Entrega Final (v1.0.0)	Revisión contra ISO/IEC/IEEE 29148:2018 (M5–M21): estados y rutas al día con el código en inglés, campo Método de verificación en cada RF, esquema inventario en §2.1, RF-11b como decisión ética abierta, fila RF-36 (módulo de equipos, Planificado), columna estado de la matriz normalizada al vocabulario del §1.3, secciones nuevas §4.5 Interfaces externas , §4.6 Máquinas de estado , §4.7 Matriz de permisos y §4.8 correspondencia con el Anexo C . Adiciones (A21, A22): RNF-14 política de contraseñas unificada, RF-37 restablecimiento de contraseña por enlace y RNF-15 correo saliente (matriz de trazabilidad: 53 filas).

1.5	2026-09-07	Entrega Final (v1.0.0)	Cierra los puntos A1–A20 de la misma revisión: se especifican 11 RF de código ya construido sin requisito — \$3.5 (RF-38 pagos, RF-39 consentimiento, RF-40 informes al representante, RF-41 representantes como recurso, RF-42 consulta de auditoría, RF-43 reportes en PDF, RF-44 exportación de datos propios, RF-45 alertas, RF-46 catálogos especialidad/posición, RF-47 resumen/autoconsulta de asistencia, RF-48 observaciones de texto libre) — y 9 RNF: RNF-16 frontera de datos al LLM, RNF-17 protección de datos de menores, RNF-18 usabilidad SUS y RNF-19 accesibilidad (nueva \$4.9), RNF-20 quality gate SonarQube, RNF-21 certificado TLS de producción, RNF-22 conservación y supresión, RNF-23 indisponibilidad de Redis, RNF-24 respaldo y recuperación (matriz: 73 filas).
1.6	2026-09-08	Entrega Final (v1.0.2)	Revisión M1–M9: matriz corregida (RF-38/RF-43 con comas entrecomilladas, división RF-19a/RF-19b, columna observaciones, estados solo del vocabulario Implementado/Modelado/Planificado, retirada la fila huérfana RF-36), citas de clases de prueba y controladores al día con el código en inglés, Método de verificación con vocabulario cerrado {Test, Demostración, Análisis, Inspección} en los 73 requisitos, plantilla uniforme del módulo deportivo (títulos sin estado, campo Estado en la línea Prioridad), decisión RF-48 (M7) documentada y cabecera con el commit a defender. Puntos A3 y A4 de la revisión de septiembre: RNF-23 dividido en RNF-23a (autenticación falla-cerrado, Implementado) y RNF-23b (degradación de la caché de listados con <code>CacheErrorHandler</code>, Planificado, con criterio y condición de cierre); RNF-24 reforzado con destino de almacenamiento fijado, PITR del proveedor declarado, RPO ≤ 24 h explícito y evidencia archivada de restauración cronometrada. Punto A2: los hallazgos de <code>ETHICS.md</code> pasan de riesgo declarado a requisito con criterio de cierre — nueva \$3.6 con RF-49 (H-01, cédula opcional y validada), RF-50 (H-03, supresión/anonimización), RF-51 (H-04/H-07, consentimiento como compuerta del envío) y RNF-25 (H-02, control del texto libre); RNF-17 reescrito como paraguas con la tabla hallazgo→requisito. Cierre A1: el validador comprueba que toda ruta de archivo citada en el SRS exista en disco (detectó y corrigió la cita de un diseño IA inexistente y <code>nginx/default.conf</code>→<code>frontend/nginx.conf</code>); RNF-23b con fecha objetivo fijada (2026-09-15, antes de la defensa). M7 decidido (2026-09-08): RF-11b (peso y altura) se conserva con finalidad, base legal (consentimiento del representante, alcance físico-deportivo, LOPDP) y conservación documentadas; cierra el hallazgo H-06. Implementación (2026-09-08): RNF-23b (<code>CacheErrorHandler</code> en <code>RedisCacheConfig</code>), RNF-25 completo (topes de longitud en servidor — <code>@Size</code> de lesión y asistencia, guarda en <code>EvaluacionDiariaService.finalizar</code> — y a nivel de motor — <code>v25__limite_texto_libre_menores.sql</code>—, control de acceso ya restringido, y guía de redacción en el formulario de lesión de la pantalla de evaluación diaria con contador y <code>maxLength</code>), RF-11b/H-06 (lectura de peso/altura restringida a ADMINISTRADOR/ENTRENADOR en <code>StudentController</code>) y RF-51 (ya estaba: <code>NotificationService</code> consulta el consentimiento antes de crear la notificación) → los cuatro pasan a  Implementado y RF-48 sale de MoSCoW Won't. y RF-49 (cédula opcional + validación de dígito verificador <code>@Cedula</code> + índice único parcial v26; las

		cédulas de seed se cargan por SQL directo y no se validan) → ☒ Implementado. RF-50 (supresión / anonimización): procedimiento almacenado versionado academico.sp_anonimizar_estudiante (migración V27) + POST /api/estudiantes/{id}/anonimizar restringido a ADMINISTRADOR y auditado (@Audited "ANONIMIZAR") — sustituye los datos identificativos de la persona por valores neutros, borra el texto libre sobre el menor y da de baja lógica la ficha, conservando FKs y agregados → ☒ Implementado. Con esto cierran H-01, H-02, H-03 y H-04, y RF-48 sale de MoSCoW Won't.
--	--	---

7. Aprobación

Este documento constituye la especificación de requisitos acordada para la Entrega Final del proyecto SGED, cerrada en la etiqueta **v1.0.0** del repositorio.

Rol	Nombre	Firma	Fecha
Autor (equipo)	Arcalle Grefa Darwin Orlando		2026-09-04
Autor (equipo)	Pallo Pinto Alejandro Daniel		2026-09-04
Autor (equipo)	Velez Lopez Ricardo Elias		2026-09-04
Docente evaluador	Dr. Gleiston Cicerón Guerrero Ulloa, Ph.D.	_____	_____